

11 代理模式

控制对象访问

有你做我的代理，我就可以从朋友那里抽到三倍的午餐钱了！



玩过扮黑脸扮白脸的游戏吗？你是一个白脸，你提供服务的方式都是和蔼友善的，但是你不希望每个人都叫你做事，因此找了黑脸控制对你的访问。这就是代理要做的：控制和管理访问。就像你将看到的，挡在所代理的对象前面的代理有很多种方式。代理以通过Internet搬运被代理对象的整个方法调用而出名。另外一个出名的事情是，它耐心地守护某些很懒惰的对象。



还记得万能糖果公司的
CEO吗？

各位组员，我真的希望我的糖果机能够获得更好的监控，你能想办法给我一份库存以及机器状态的报告吗？

听起来够容易的。如果你还记得，在糖果机代码中，我们已经有`getCount()`方法取得糖果数量，以及`getState()`方法取得糖果机当前状态。

我们要做的，就是创建一份能打印出来的报告送给CEO。嗯，我们可能需要为每个糖果机加上一个位置字段，这样CEO就可以一目了然。

让我们现在就开始编码。一定会让CEO印象深刻，迅速对我们刮目相看。

为监视器编码

我们先为GumballMachine类加上处理位置的支持：

```
public class GumballMachine {
    // 其他实例变量
    String location;

    public GumballMachine(String location, int count) {
        // 构造器的其他代码
        this.location = location;
    }

    public String getLocation() {
        return location;
    }

    // 其他方法
}
```

位置是一个String。

位置被传入构造器，然后存在此实例变量中。

让我们也加上一个getter方法，以便需要时获取位置。

现在我们创建另一个类GumballMonitor（糖果监视器）。这个类检索机器的位置、糖果的库存，以及当前的机器状态，并打印成一份小巧可爱的报告。

```
public class GumballMonitor {
    GumballMachine machine;

    public GumballMonitor(GumballMachine machine) {
        this.machine = machine;
    }

    public void report() {
        System.out.println("Gumball Machine: " + machine.getLocation());
        System.out.println("Current inventory: " + machine.getCount() + " gumballs");
        System.out.println("Current state: " + machine.getState());
    }
}
```

监视器的构造器需要传入糖果机作为参数，然后把它赋值给machine实例变量。

report()方法打印报告，报告中有位置、库存和机器状态。

测试监视器

我们一下就实现好了。CEO被我们的开发技艺震住了。

现在，我们需要实例化一个GumballMonitor（糖果监视器）并给它一个糖果机。

```
public class GumballMachineTestDrive {
```

```
    public static void main(String[] args) {
        int count = 0;
```

通过命令行传入位置和
初始糖果数目。

```
        if (args.length < 2) {
            System.out.println("GumballMachine <name> <inventory>");
            System.exit(1);
        }
```

别忘了把位置和数目
给构造器……

```
        count = Integer.parseInt(args[1]);
        GumballMachine gumballMachine = new GumballMachine(args[0], count);
```

```
        GumballMonitor monitor = new GumballMonitor(gumballMachine);
```

……然后实例化一个监视器，传给
它要提供报告的糖果机。

```
        // rest of test code here
```

```
        monitor.report();
```

当我们需要糖果机的报告时，
调用report()方法。

File Edit Window Help FlyingFish

```
%java GumballMachineTestDrive Austin 112
```

```
Gumball Machine: Austin
```

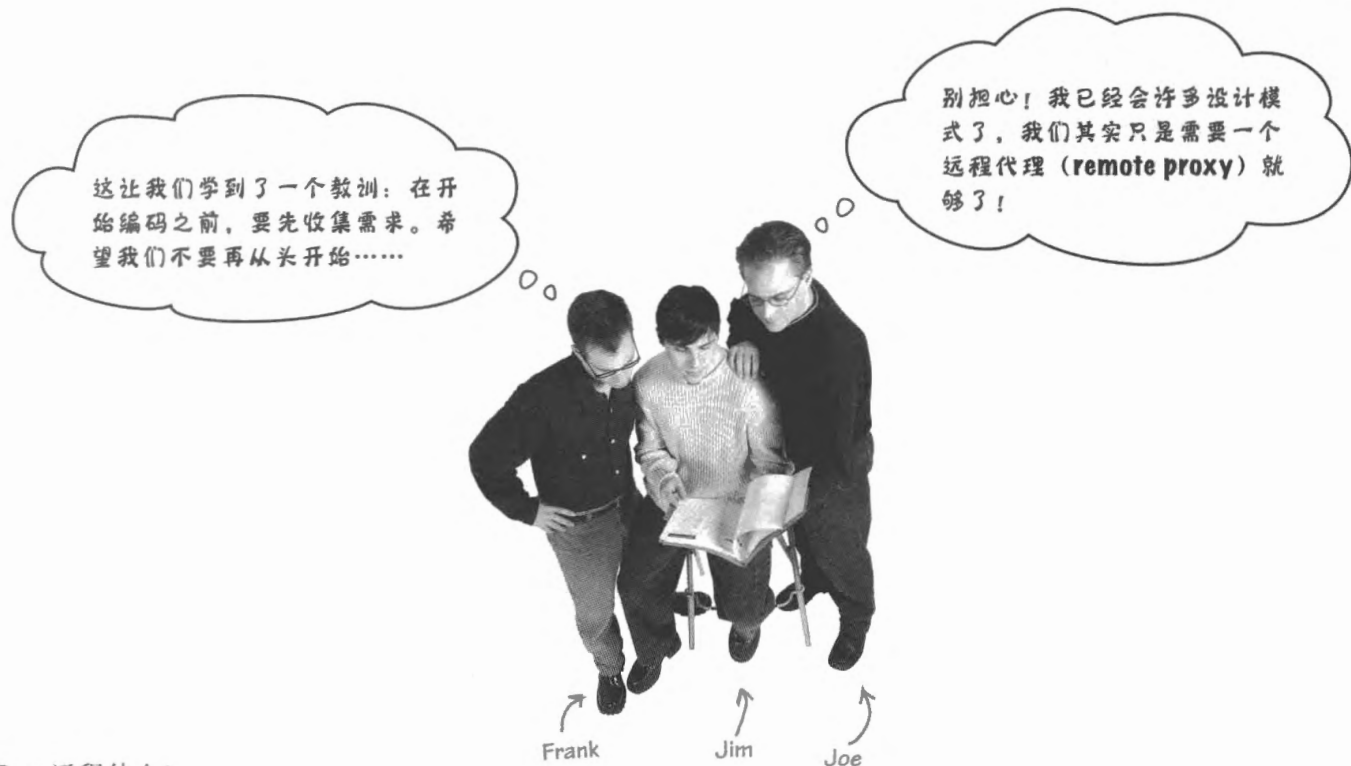
```
Current Inventory: 112 gumballs
```

```
Current State: waiting for quarter
```

输出在这里！



监视器的输出看起来很棒，但可能之前我
没说清楚。我需要的是远程监控糖果机！
事实上，我们已经把网络准备好了。拜托，
你们可是号称互联网的一代！



Joe: 远程什么？

Frank: 远程代理。想想：我们已经写好监视器代码，对吧？我们给GumballMonitor类一个糖果机的引用，它给我们一份报告。问题在于监视器和糖果机在同一个JVM上运行，但CEO要的是坐在办公桌前远程监控这些机器！这样怎么样？我们保持GumballMonitor类不变，但交给它的是一个远程对象的代理。

Joe: 我不太懂。

Jim: 我也是。

Frank: 让我从头开始说……代理（proxy）代表一个真实的对象。在这个例子中，代理表现得就像糖果机对象一样，但其实幕后是通过网络和一个远程的真正糖果机沟通。

Jim: 你是说，我们的代码保留原样，只要将GumballMachine代理版本的引用交给监视器就可以……

Joe: 然后这个代理假装它是真正的对象，但实际上它只是通过网络和真正的对象沟通。

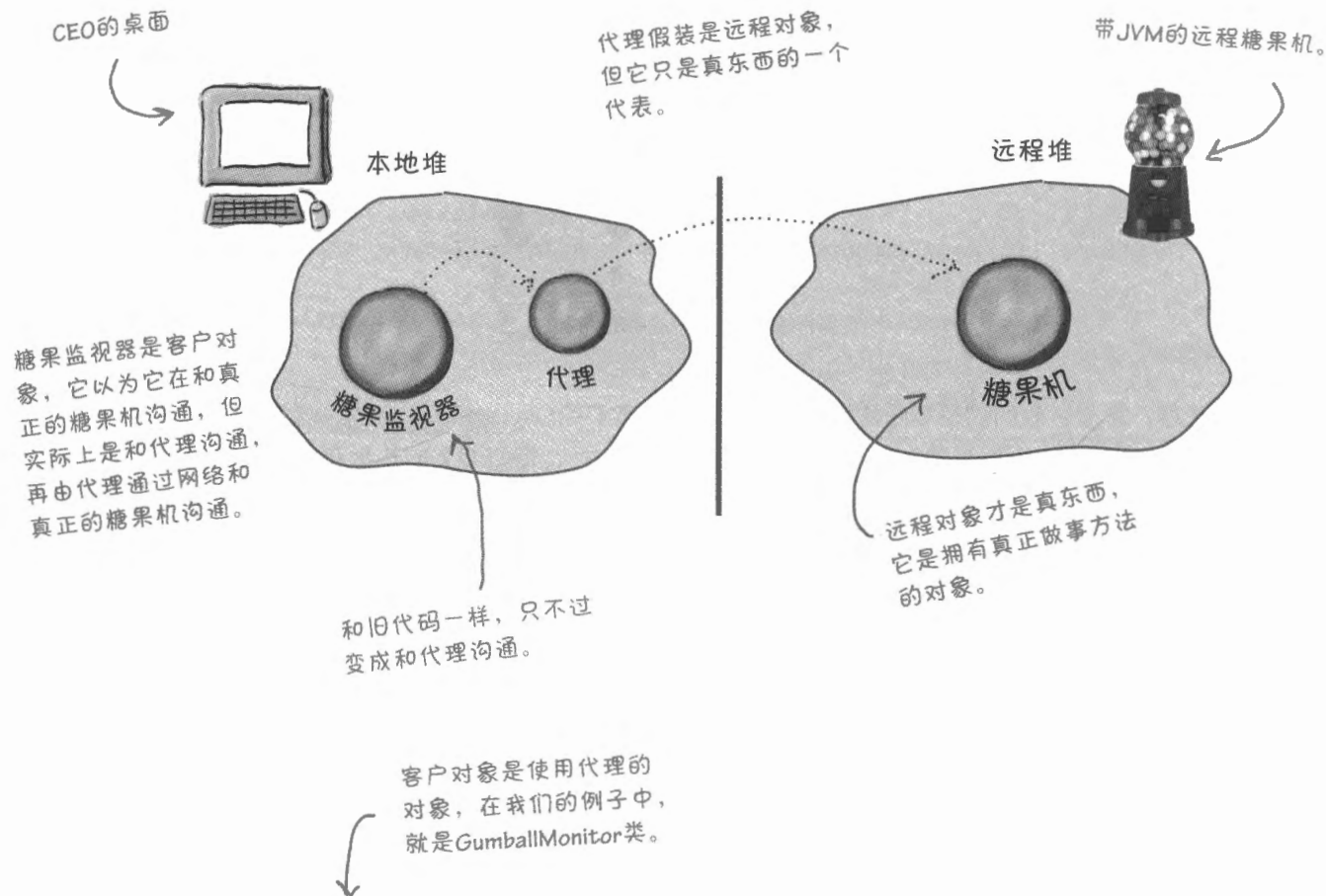
Frank: 嗯，差不多就是这样。

Joe: 这事好像说起来比做起来容易。

Frank: 或许吧！但是我不认为有这么难。我们必须确定糖果机能够通过网络接受请求并且提供服务，也需要让监视器有办法取得代理对象的引用，这方面，幸好Java已经有一些很棒的内置工具可以帮助我们。我们先看看远程代理的更多内容……

远程代理的角色

远程代理扮演远程对象的本地代表。什么是“远程对象”？就是活在不同的Java虚拟机堆中的对象（或者说得更通用些，远程对象运行在不同的地址空间）。什么是“本地代表”？就是你可以在上面调用本地方法的对象，它把调用转发给远程对象。



你的客户对象表现得好像是在做远程方法调用，但其实只是调用本地堆中的“代理”对象上的方法，代理对象处理所有网络通信的低层细节。

真是个好主意。我们要写一些方法调用的代码，反正调用后会通过网络传送，调用远程对象的一些方法。然后我假设，当调用完成时，结果通过网络送回我们的客户。但看起来，这样的代码对我来说很难写。



等等，我们没有打算自己写这些代码，**Java**已经内建远程调用的功能了，我们只需要翻新一下代码，以便利用**RMI**的优势。

BRAIN POWER

在我们往前走之前，想想看要如何设计一个系统，支持远程方法调用。你怎样才能让这件事变得容易，这样开发人员就可以尽可能少地写代码？你怎样才能让远程调用看起来像本地调用一样，让人看不出毛病？

BRAIN² POWER

远程调用程序应该完全透明吗？这是个好主意吗？这个方法可能会产生问题吗？

将远程代理加到糖果机的监视代码中

纸面上，我们的计划看起来不错，但是我们怎样创建一个代理，知道如何调用活在另一个JVM中的对象的方法？

嗯……你不能取得另一个堆的对象的引用，对吧？换句话说，你不能说：

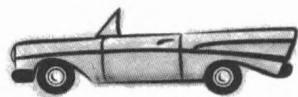
Duck d = <另一个堆的对象>

不管引用什么，变量d引用的必须是和运行语句的代码在相同堆空间的东西。那该怎么办？嗯，该是Java远程方法调用（RMI）上场的时刻了……RMI可以让我们找到远程JVM内的对象，并允许调用它们的方法。

现在可能是一个好时机，拿出你喜欢的Java参考资料温习一下RMI，或者你可以先看“RMI浏览”，我们将为你介绍一下RMI的进阶知识，然后我们再为糖果机代码添加代理支持。

我们的计划如下：

- ❶ 首先，我们先浏览和探索一下RMI。即使你熟悉RMI，你可能还想复习顺便跟着浏览一下风景。
- ❷ 接着，我们会把GumballMachine变成远程服务，提供一些可以被远程调用的方法。
- ❸ 最后，我们将创建可以和远程糖果机沟通的代理，再次用到RMI，并和监视系统整合到一起。这样，CEO就可以监视任意数量的远程糖果机了。



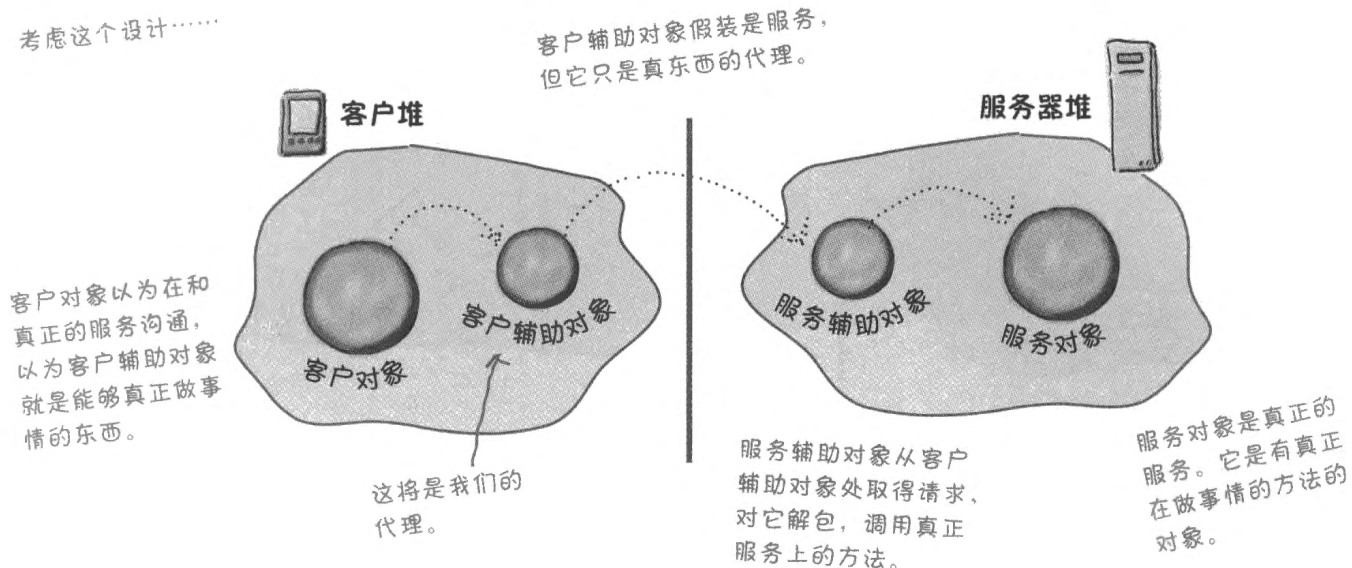
RMI浏览

如果你是RMI新手，浏览下面几页。否则，匆匆扫一下就可以了。如果你想继续往下走，了解远程代理的要点就行，其他可以跳过。



远程方法101

考虑这个设计……



走一遍设计

假如我们想要设计一个系统，该系统允许我们调用本地对象，然后本地对象将每个请求转发给远程对象。要如何设计？我们需要一些辅助对象帮我们做这些沟通。这些辅助对象能够让客户像在调用本地对象的方法（事实也是如此）一样。客户调用客户辅助对象上的方法，仿佛客户辅助对象就是真正的服务。客户辅助对象再负责为我们转发这些请求。

换句话说，客户对象以为它正在调用远程服务上的方法，因为客户辅助对象乔装成服务对象。也就是说，假装自己有客户要调用的方法。

但是客户辅助对象不是真正的远程服务，虽然扮演起来很像（因为它有服务所宣称的相同的方法）。客户辅助对象没有任何客户所期望的方法逻辑。客户辅助对象会联系服务器，传送方法调用信息（例如，方法名称、参量等），并等待服务器的返回。

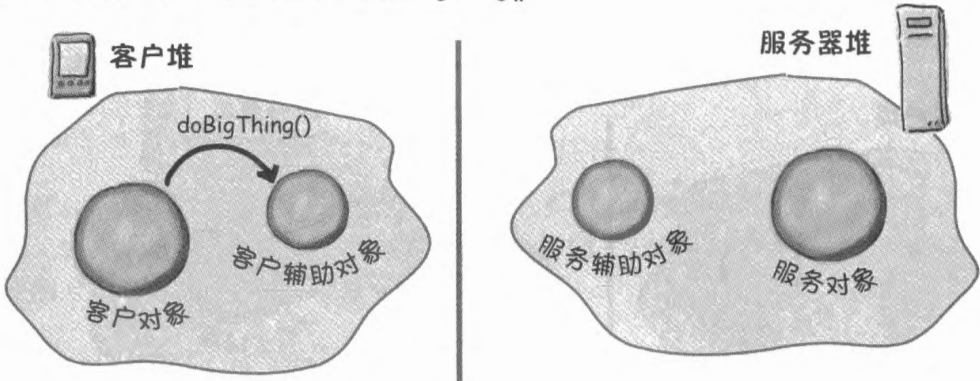
在服务器端，服务辅助对象从客户辅助对象处接收请求（通过Socket连接），将调用的信息解包，然后调用真正服务对象上的真正方法。因此，对于服务对象来说，调用是本地的，来自服务辅助对象，而不是远程客户。

服务辅助对象从服务得到返回值，将它打包，然后运回到客户辅助对象（通过Socket的输出流），客户辅助对象把信息解包，返回值给客户对象。

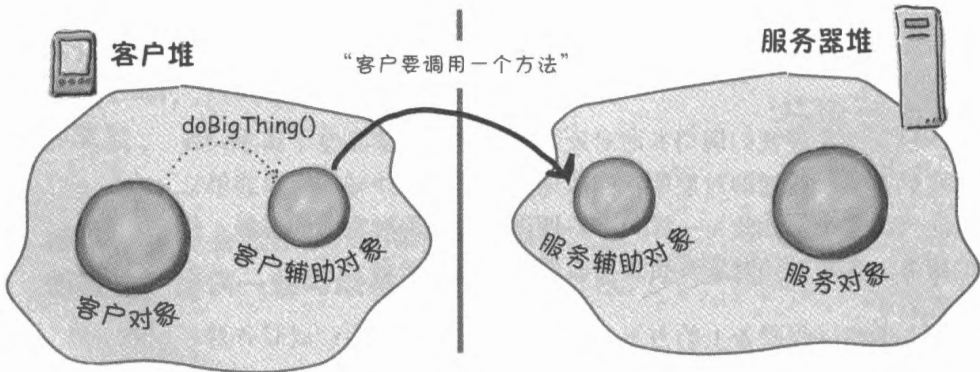
我们来走一遍会更清晰……

方法调用如何发生

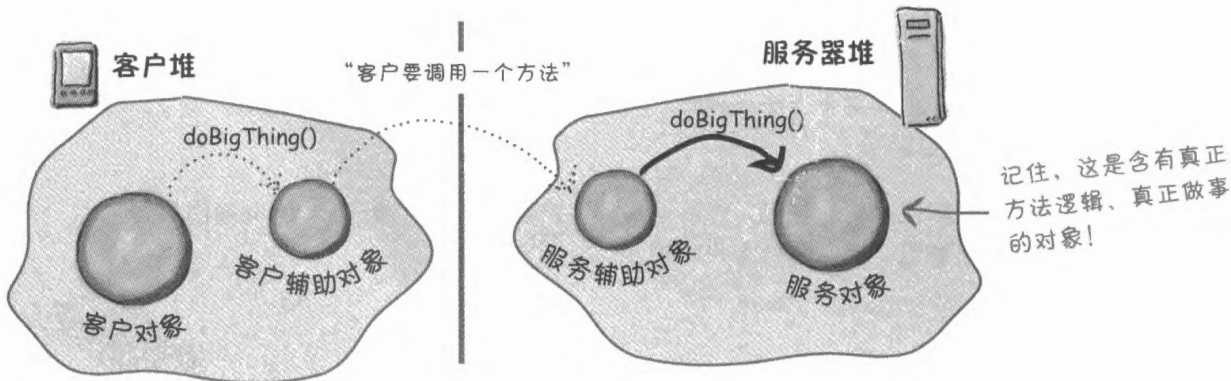
① 客户对象调用客户辅助对象上的doBigThing()。



② 客户辅助对象打包调用信息（参量、方法名称等），并通过网络运送给服务辅助对象。

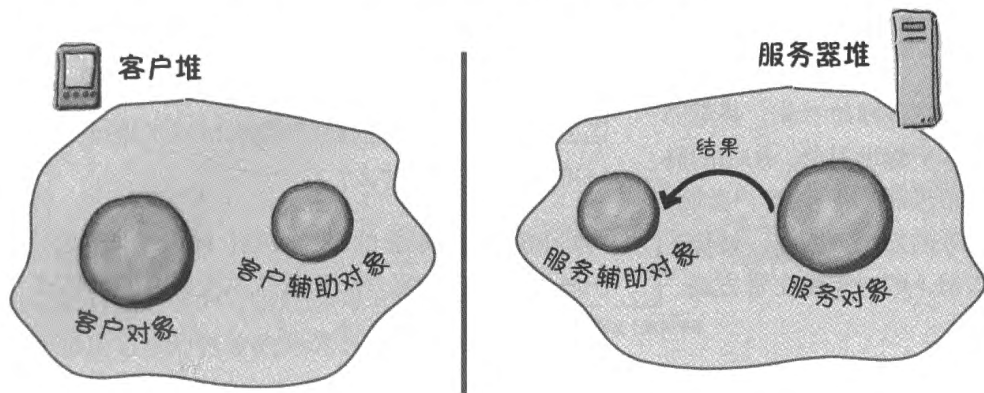


③ 服务辅助对象解包来自客户辅助对象的信息，找出被调用的方法（以及在哪个对象上），并调用真正的服务对象上的真正方法。

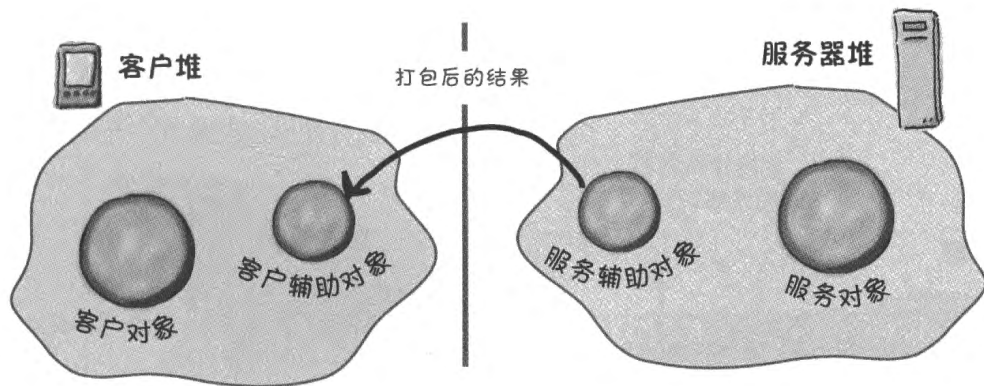




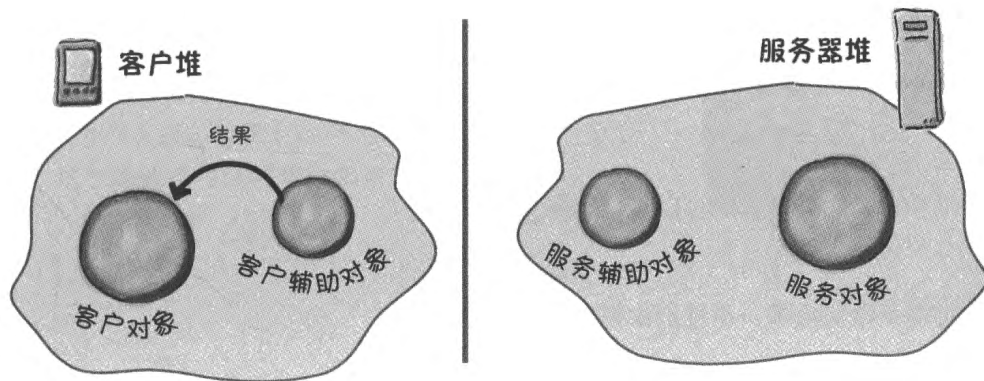
- ④ 服务对象上的方法被调用，返回一些结果给服务辅助对象。



- ⑤ 服务辅助对象把调用的返回信息打包，然后通过网络运回给客户辅助对象。



- ⑥ 客户辅助对象把返回值解包，返回给客户对象。对于客户对象来说，这是完全透明的。



Java RMI概况



OK，现在你已经知道远程方法如何运作的要点，你还需要了解如何使用RMI。

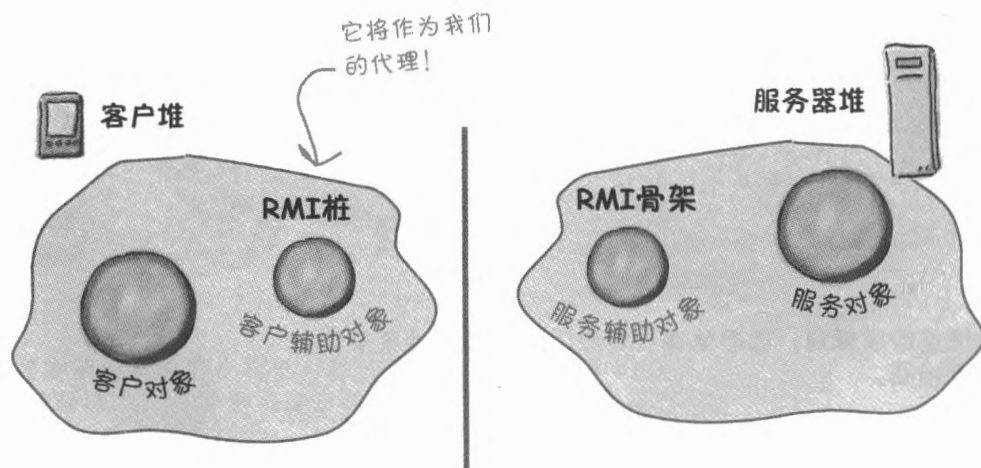
RMI所做的事情是建造客户和服务辅助对象，甚至创建和远程服务有相同方法的客户辅助对象。RMI的好处在于你不必亲自写任何网络或I/O代码，客户会调用远程方法（即真正拥有服务的那个东西），就像对运行在客户自己的本地JVM上的对象做正常方法调用一样。

RMI也提供了所有运行时的基础设施，好让这一切正常运作，包括查找（lookup）服务，这个服务用来寻找和访问远程对象。

RMI调用和本地（正常的）的方法调用有一个不同点。别忘了，即使对客户来说，RMI调用看起来和本地方法调用一样，但是客户辅助对象是通过网络发送方法调用的。因此存在网络和I/O的问题。网络和I/O的方法意味着什么？

网络和I/O是有风险的，会失败！随时随地都可能抛出异常。因此，客户必须承认风险的存在。再过几页我们就会讨论这部分。

RMI命名：RMI将客户辅助对象称为**stub（桩）**，服务辅助对象称为**skeleton（骨架）**。



现在，我们就来看看如何一步步把一个对象变成可以接受远程调用的服务，以及如何允许客户做远程调用。

接下来会有很多步骤……你可得系紧安全带，不过别太担心！

制作远程服务

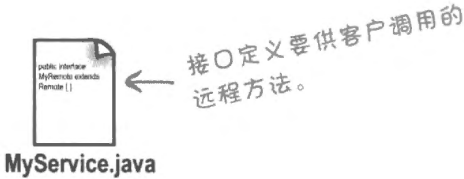


这里有五个步骤概要，用来制作远程服务。换句话说，这些步骤需要把一个普通的对象“增压”，让它可以被远程客户调用。我们稍后会把这些步骤用到糖果机。现在，就让我们一步步往下看，逐一解释细节。

步骤一：

制作远程接口。

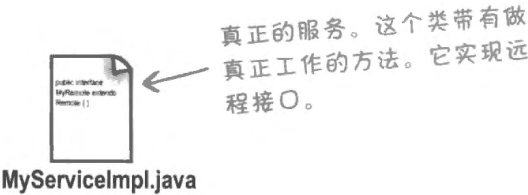
远程接口定义客户可以远程调用的方法。客户将用它作为服务的类类型。桩和实际的服务都实现此接口。



步骤二：

制作远程实现。

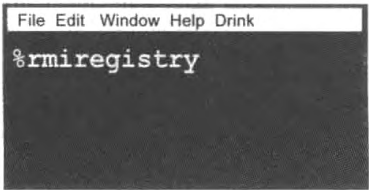
这是做实际工作的类。它有在远程接口中定义的远程方法的真正实现。这就是客户真正想要调用方法的对象（例如GumballMachine）。



步骤三：

启动RMI registry (rmiregistry)。

rmiregistry就像电话簿的白页，客户可以从中找到代理（客户的桩/辅助对象）。

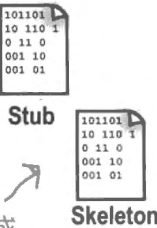
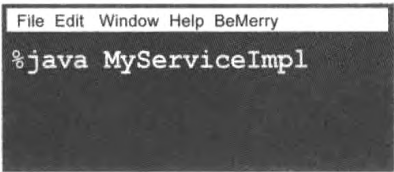


在分离的终端窗口执行这个。

步骤四：

开始远程服务。

你必须让服务对象开始执行。你的服务实现类实例化一个服务的实例，并把它注册到RMI registry。注册之后，这个服务就可以供客户调用。



在后台动态生成Stub和Skeleton。

步骤一：制作远程接口



① 扩展java.rmi.Remote。

Remote是一个“记号”接口，意思是它没有方法。不过，它于RMI来说有特别的含义，因此你必须遵守这条规则。注意，我们这里说“扩展”（extends）。一个接口可以扩展另一个接口。

```
public interface MyRemote extends Remote {
```

这告诉我们，该接口要用来支持远程调用。

② 声明所有的方法都抛出RemoteException。

远程接口是客户作为服务的类型使用的。换句话说，客户调用实现远程接口的某些东西上的方法，这些东西就是桩，当然，因为桩做网络和I/O方面的事情，因此各种坏事情都可能会发生。客户必须承认风险的存在，通过处理或声明远程异常来解决。如果接口中的方法声明了异常，任何在该类型（接口类型）的引用上调用方法的代码必须处理或声明异常。

```
import java.rmi.*;
```

← 远程接口在java.rmi中。

```
public interface MyRemote extends Remote {
    public String sayHello() throws RemoteException;
}
```

每次远程方法调用都必须考虑成“有风险的”。在每个方法中声明RemoteException，强制客户注意并承认事情可能会出错。

③ 确定参量和返回值是属于原语（primitive）类型或Serializable类型。

远程方法的参量和返回值，必须是原语类型或者Serializable类型。想想看，远程方法的任何参量都必须被打包并通过网络运送，这要靠序列化来完成。返回值也是一样。如果你使用原语类型、字符串，以及许多API中存在的类型（包括数组和集合），不会有问题。如果你传送自己的类型，就必须保证你的类实现了Serializable。

如果你需要复习一下Serializable，可以看看你最喜欢的Java参考资料。

```
public String sayHello() throws RemoteException;
```

← 这个返回值将从服务器经过网络运回给客户，因此必须是Serializable的。这样，参量和返回值才能被打包和发送。



步骤二: 制作远程实现

① 实现远程接口。

你的服务必须实现远程接口，带有客户要调用的方法的接口。

```
public class MyRemoteImpl extends UnicastRemoteObject implements MyRemote {
    public String sayHello() {
        return "Server says, 'Hey'";
    }
    // 类中更多的代码
}
```

← 编译器会确认你实现此接口所有的方法。此例中，只有一个方法。

② 扩展UnicastRemoteObject。

为了成为远程服务对象，你的对象需要某些和“远程”有关的功能，最简单的方式是扩展UnicastRemoteObject（来自java.rmi.server包），让这个类（你的超类）做这些工作。

```
public class MyRemoteImpl extends UnicastRemoteObject implements MyRemote {
    private static final long serialVersionUID = 1L;
}
```

← UnicastRemoteObject实现Serializable，所以我们需要serialVersionUID字段。

③ 编写一个声明RemoteException的无参量构造器。

你的新超类，UnicastRemoteObject，有一个小问题，它的构造器抛出RemoteException。解决这个问题的唯一方法是为你的远程实现声明一个构造器，这样就有了一个地方来声明RemoteException。记住，当类被实例化时，其超类的构造器总是会被调用。如果超类的构造器抛出一个异常，那么你没有选择，你的构造器也要抛出一个异常。

```
public MyRemoteImpl() throws RemoteException { }
```

你不必在构造器中放任何东西，只需要有办法声明超类构造器抛出异常。

④ 用RMI registry注册服务。

现在你有了一个远程服务，必须让它可以被远程客户调用。要做的是实例化它并放进RMI Registry（RMI Registry必须正在运行，否则注册会失败）。当注册这个实现对象时，RMI系统其实注册的是桩，因为这是客户真正需要的。注册服务是用java.rmi.Naming类的静态rebind()方法。

```
try {
    MyRemote service = new MyRemoteImpl();
    Naming.rebind("RemoteHello", service);
} catch (Exception ex) { ... }
```

为你的服务起名（好让客户用来在注册表中查找它），并在RMI Registry中注册。当你绑定（bind）服务对象时，RMI会把服务换成桩，然后把桩放进注册表。

步骤三: 运行rmiregistry



① 开启一个终端，启动 rmiregistry。

先确定启动目录必须可以访问你的类。最简单的做法是从你的“classes”目录启动。

File Edit Window Help Huh?

```
%rmiregistry
```

步骤四: 启动服务

① 开启另一个终端，启动服务。

可以从你的远程实现类中的main()方法启动服务，或者从一个分离的启动类。在这个简单的例子中，我们把启动器的代码放进实现类的main()方法。该方法实例化对象并在RMIRegistry中注册。

File Edit Window Help Huh?

```
%java MyRemoteImpl
```

没有蠢问题

问：为什么要在RMI代码的图中显示桩和骨架？我想可以不用。

答：你说对了；对于骨架来说，RMI runtime可以使用反射把客户的调用直接分派给远程服务，桩是使用动态代理生成的（更多内容本章后面一点会提到）。远程对象的桩是一个java.lang.reflect.代理实例（带有一个调用处理器）自动生成，取得客户发起的本地方法调用并发给远程对象。但我们喜欢展示桩和骨架，因为在概念上，这样做帮助理解客户桩和远程服务之间的沟通背后的秘密。



服务器端的完整代码

我们看看服务器端的所有代码：

远程接口：

```
import java.rmi.*;
```

RemoteException和远程接口在java.rmi包中。

```
public interface MyRemote extends Remote {
```

你的接口必须扩展java.rmi.Remote。

```
    public String sayHello() throws RemoteException;
```

所有的远程方法都必须声明RemoteException。

```
}
```

远程服务 (实现)：

```
import java.rmi.*;
```

UnicastRemoteObject在java.rmi.server包中。

```
import java.rmi.server.*;
```

要制作一个远程对象，扩展UnicastRemoteObject是最容易的方法。

```
public class MyRemoteImpl extends UnicastRemoteObject implements MyRemote {
```

当然，你必须实现所有的接口方法，但注意，不必声明RemoteException。

```
    private static final long serialVersionUID = 1L;
```

你必须实现你的远程接口！

```
    public String sayHello() {
```

```
        return "Server says, 'Hey'";
```

```
    }
```

你的超类 (UnicastRemoteObject) 构造器声明了一个异常，因此你必须写一个构造器，因为这意味着你的构造器正在调用有风险的代码 (它的超类构造器)。

```
    public MyRemoteImpl() throws RemoteException { }
```

制作远程对象，然后使用静态的Naming.rebind() 把它“绑定”到rmiregistry。客户将使用你注册的名称在RMI Registry中查找。

```
    public static void main (String[] args) {
```

```
        try {
```

```
            MyRemote service = new MyRemoteImpl();
```

```
            Naming.rebind("RemoteHello", service);
```

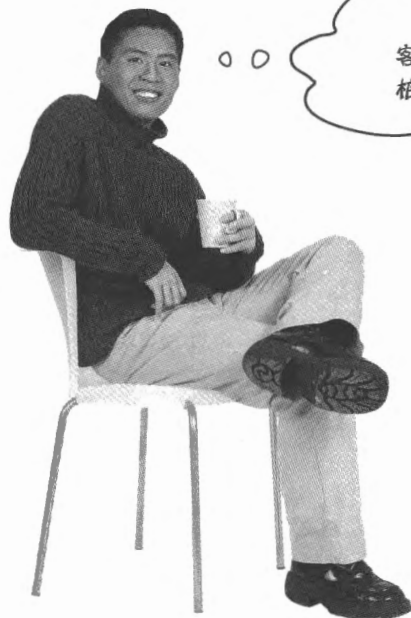
```
        } catch (Exception ex) {
```

```
            ex.printStackTrace();
```

```
        }
```

```
    }
```

```
}
```



〇〇

客户实际上是如何取得
桩对象的？

这里就是用到RMI registry的地方。

你说得没错，客户必须取得桩对象（我们的代理），因为客户要调用它上面的方法。为了达到这个目的，客户像在电话簿白页一样做“查找”，基本上是说，“这里有个名称，我要找这个名称的桩。”

我们来看看查找和检索桩对象的代码。

关于这是怎么工作的，
下一页会讲解。

Code up Close

客户总是使用远程接口作为服务类型，事实上，客户不需要知道远程服务的实际类名。

↓
MyRemote service =

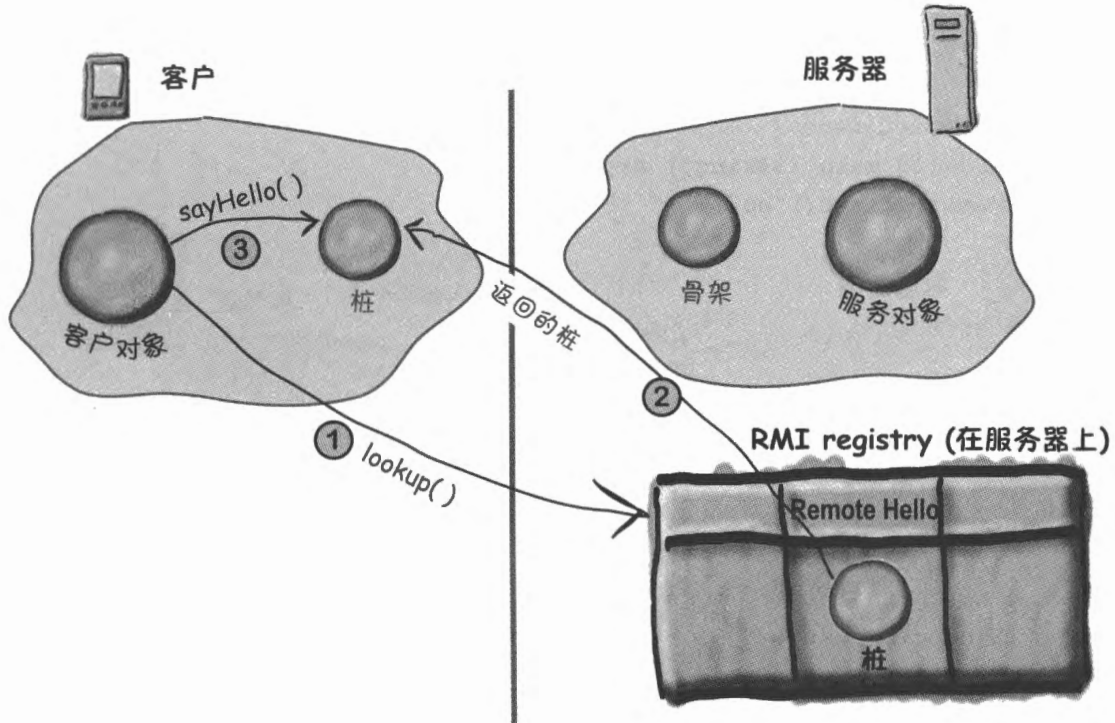
(MyRemote) Naming.lookup("rmi://127.0.0.1/RemoteHello");

↑
lookup()方法返回Object类型，你必须把它转成接口。

lookup()是Naming类的静态方法。

↓
这必须是服务注册时用的名称。

↑
服务运行的主机名或者IP地址（127.0.0.1是本地主机）。



工作机制……

- ① 客户在RMI registry中查找。
`Naming.lookup("rmi://127.0.0.1/RemoteHello");`
- ② RMI registry返回桩对象。
 (作为lookup方法的返回值) RMI自动把桩反序列化。
- ③ 客户调用桩上的方法，就像桩是真正的服务一样。

客户端的完整代码



我们来看看客户端的所有代码：

```
import java.rmi.*;

public class MyRemoteClient {
    public static void main (String[] args) {
        new MyRemoteClient().go();
    }

    public void go() {
        try {
            MyRemote service = (MyRemote) Naming.lookup("rmi://127.0.0.1/RemoteHello");

            String s = service.sayHello();

            System.out.println(s);
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }
}
```

Naming类（用来做rmiregistry查找）
在java.rmi包中。

返回的是Object类型，因此别忘了
转换类型。

你需要IP地址或者
主机名。

……以及用于绑定/重绑定
服务时的名称。

看起来和常规的老式方法
调用一样（除了必须注意
RemoteException之外）。



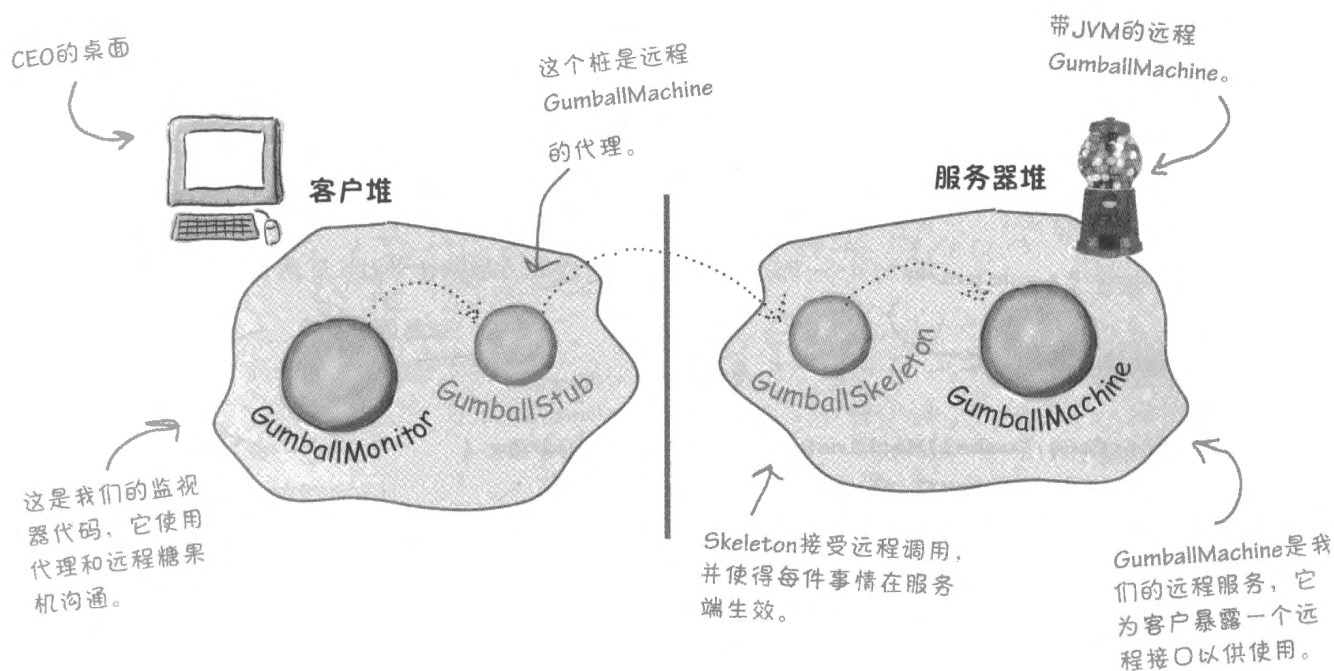
对于RMI，程序员最常犯的错误：

1. 忘了在启动远程服务之前先启动rmiregistry（要用Naming.rebind()注册服务，rmiregistry必须是运行的）！
2. 忘了让参量和返回类型可序列化（这种错误编译不能侦测，直到运行时才会知道）。



回到我们的GumballMachine远程代理

OK，现在你有了RMI的基础，也就有了实现糖果机远程代理所需的工具。我们来看看GumballMachine如何套进RMI框架：



停下来想一想，我们该怎样修改代码以便让糖果机可以通过远程代理工作。请记下来哪些需要改变，哪些地方与以往版本不一样。

让GumballMachine准备好成为一个远程服务

要把我们的代码转变成使用远程代理，第一个步骤是让GumballMachine变成可以接受来自客户的远程调用。换句话说，把它变成一个服务。做法如下：

1. 为GumballMachine创建一个远程接口。接口提供一组可以远程调用的方法。
2. 确定接口的所有返回类型都是可序列化的。
3. 在一个具体类中实现该接口。

我们从远程接口开始：

别忘了 import java.rmi.*。

```
import java.rmi.*;

public interface GumballMachineRemote extends Remote {
    public int getCount() throws RemoteException;
    public String getLocation() throws RemoteException;
    public State getState() throws RemoteException;
}
```

这是远程接口。

所有的返回类型都必须
是原语类型或可序列化
类型……

这是准备支持的方法，每个都抛出
RemoteException。

我们有一个返回类型不是可序列化的：State类。我们来把它修正过来……

```
import java.io.*;

public interface State extends Serializable {
    public void insertQuarter();
    public void ejectQuarter();
    public void turnCrank();
    public void dispense();
}
```

Serializable在java.io包内。

然后我们只要扩展 Serializable接口（里面
没有方法）。现在所有子类中的State就
可以在网络上传送。

实际上，我们还没处理完Serializable。对于State，我们有一个问题。你可能记得，每个状态对象维护一个到糖果机的引用，因此它可以调用糖果机的方法并改变糖果机的状态。我们不希望整个糖果机都被序列化并随着State对象一起传送。修正这点很容易：

```
public class NoQuarterState implements State {
    private static final long serialVersionUID = 2L;
    transient GumballMachine gumballMachine;
    // 其他方法在这里。
}
```

在State的每个实现中，我们给GumballMachine实例变量加上serialVersionUID以及transient关键字。transient关键字告诉JVM不要序列化这个字段。注意，一旦对象被序列化并传送，如果你尝试访问这个字段，会有点危险。

我们已经实现了GumballMachine，但是需要确定它可以扮演服务的角色，处理来自网络的请求。为了做到这一点，我们必须确定GumballMachine正在做实现GumballMachineRemote接口所需的每一件事情。

你已经在RMI浏览里看到了，这十分简单，我们只需要加一些东西……

首先我们需要import rmi包。

```
import java.rmi.*;
import java.rmi.server.*;
```

GumballMachine要继承UnicastRemoteObject，这给了它成为远程服务的能力。

GumballMachine也需要实现这个远程接口。

```
public class GumballMachine
    extends UnicastRemoteObject implements GumballMachineRemote
{
    private static final long serialVersionUID = 2L;
    // 其他实例变量在这里

    public GumballMachine(String location, int numberGumballs) throws RemoteException {
        // 这里有代码
    }

    public int getCount() {
        return count;
    }

    public State getState() {
        return state;
    }

    public String getLocation() {
        return location;
    }
    // 这里有其他方法
}
```

……构造器需要抛出RemoteException，因为超类是这么做的。

不要怀疑，这里完全不用改动。

在RMI注册表中注册……

注册后，糖果机服务就算完成了。现在我们只需要把它装上去，以便接收请求。首先，需要确保它已注册到RMI注册表中，这样客户才可以找到它。

我们要添加一点代码进行测试：

```
public class GumballMachineTestDrive {

    public static void main(String[] args) {
        GumballMachineRemote gumballMachine = null;
        int count;

        if (args.length < 2) {
            System.out.println("GumballMachine <name> <inventory>");
            System.exit(1);
        }

        try {
            count = Integer.parseInt(args[1]);

            gumballMachine = new GumballMachine(args[0], count);
            Naming.rebind("//" + args[0] + "/gumballmachine", gumballMachine);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

首先，我们需要在实例化糖果的代码周围添加try/catch块，因为我们的构造器现在可能会抛出异常。

我们也添加对Naming.rebind的调用，用gumballmachine的名字发布GumballMachine桩。

开始运行……

先执行这个。

这会启动并运行RMI注册表服务。

我们用“官方”名称：你应该用自己的机器名或“本地主机”代替。

```
File Edit Window Help Huh?
% rmiregistry
```

File Edit Window Help Huh?

```
% java GumballMachineTestDrive austin.mightygumball.com 100
```

其次运行这个。

这会使得GumballMachine启动和运行，并注册到RMI注册表中。

现在轮到GumballMonitor客户……

还记得GumballMonitor吗？我们要在不重写它的情况下来复用它在网络上工作。嗯，好像有很多事要做，其实我们只需要做一些小改变。

```
import java.rmi.*;
```

← 我们需要import RMI包，因为下面用到
RemoteException类……

```
public class GumballMonitor {
    GumballMachineRemote machine;
```

现在我们准备依赖此远程接口，而不是
具体的GumballMachine类。

```
    public GumballMonitor(GumballMachineRemote machine) {
        this.machine = machine;
    }
```

```
    public void report() {
```

```
        try {
```

```
            System.out.println("Gumball Machine: " + machine.getLocation());
```

```
            System.out.println("Current inventory: " + machine.getCount() + " gumballs");
```

```
            System.out.println("Current state: " + machine.getState());
```

```
        } catch (RemoteException e) {
            e.printStackTrace();
        }
```

← 当我们试图调用那些最终要通过网络
发生的方法时，我们也需要捕获所有
可能发生的远程异常。

Joe是对的，这样的
做法相当可行。



编写监视器测试程序

现在我们已经具备所需要的一切，只需再写一些代码，让CEO可以监控一堆糖果机。

这就是监视器测试程序，CEO会运行此程序！

```
import java.rmi.*;
```

```
public class GumballMonitorTestDrive {
```

```
    public static void main(String[] args) {
```

```
        String[] location = {"rmi://santafe.mightygumball.com/gumballmachine",  
                             "rmi://boulder.mightygumball.com/gumballmachine",  
                             "rmi://austin.mightygumball.com/gumballmachine"};
```

```
        GumballMonitor[] monitor = new GumballMonitor[location.length];
```

```
        for (int i=0; i < location.length; i++) {
```

```
            try {
```

```
                GumballMachineRemote machine =
```

```
                    (GumballMachineRemote) Naming.lookup(location[i]);
```

```
                monitor[i] = new GumballMonitor(machine);
```

```
                System.out.println(monitor[i]);
```

```
            } catch (Exception e) {
```

```
                e.printStackTrace();
```

```
            }
```

```
        }
```

```
        for (int i=0; i < monitor.length; i++) {
```

```
            monitor[i].report();
```

```
        }
```

```
    }
```

```
}
```

这是所有要监视的位置。

我们创建一个位置数组，每台机器一个元素。

我们也创建一个监视器数组。

现在我们需要给每个远程机器一个代理。

然后我们遍历每台机器，打印报告。



Code up Close

这会返回一个远程糖果机的代理（如果无法定位会抛出异常）。

记住，`Naming.lookup()`是RMI包内的静态方法，它从参数中得知位置和服务名称，然后在该位置的rmiregistry中查找该名称的服务。

```
try {
    GumballMachineRemote machine =
        (GumballMachineRemote) Naming.lookup(location[i]);

    monitor[i] = new GumballMonitor(machine);
} catch (Exception e) {
    e.printStackTrace();
}
```

一旦得到远程机器的代理，我们创建一个新的GumballMonitor，把要监视的机器传给它。

为万能糖果公司CEO准备的另一个展示……

OK，是时候把所有这些放在一起进行另一个展示了。首先，确定有一些新版的糖果机正在运行新代码：

每台机器上，在后台或者从一个分离的终端窗口运行rmiregistry……

……然后运行GumballMachine，给它一个位置和初始的糖果数目。

File Edit Window Help Huh?

```
% rmiregistry &
```

```
% java GumballMachineTestDrive santafe.mightygumball.com 100
```

File Edit Window Help Huh?

```
% rmiregistry &
```

```
% java GumballMachineTestDrive boulder.mightygumball.com 100
```

File Edit Window Help Huh?

```
% rmiregistry &
```

```
% java GumballMachineTestDrive austin.mightygumball.com 250
```

畅销的机器。

接着，我们将监视器交到**CEO**手上，希望这次他会喜欢：

File Edit Window Help GumballsAndBeyond

```
% java GumballMonitorTestDrive  
Gumball Machine: santafe.mightygumball.com  
Current inventory: 99 gumballs  
Current state: waiting for quarter  
  
Gumball Machine: boulder.mightygumball.com  
Current inventory: 44 gumballs  
Current state: waiting for turn of crank  
  
Gumball Machine: austin.mightygumball.com  
Current inventory: 187 gumballs  
Current state: waiting for quarter  
%
```

监视器遍历各台远程机器，并调用它们的 `getLocation()`、`getCount()` 和 `getState()` 方法。

令人惊叹，它会彻底革新我们的业务，让我们在竞争中把对手打得一败涂地！

通过调用代理上的方法，我们使得远程调用可以跨过网络，取回字符串、整数和 `State` 对象。因为我们使用的是代理，`GumballMonitor` 根本不知道/或不在意调用是远程的（除了不得不操心要处理远程异常）。



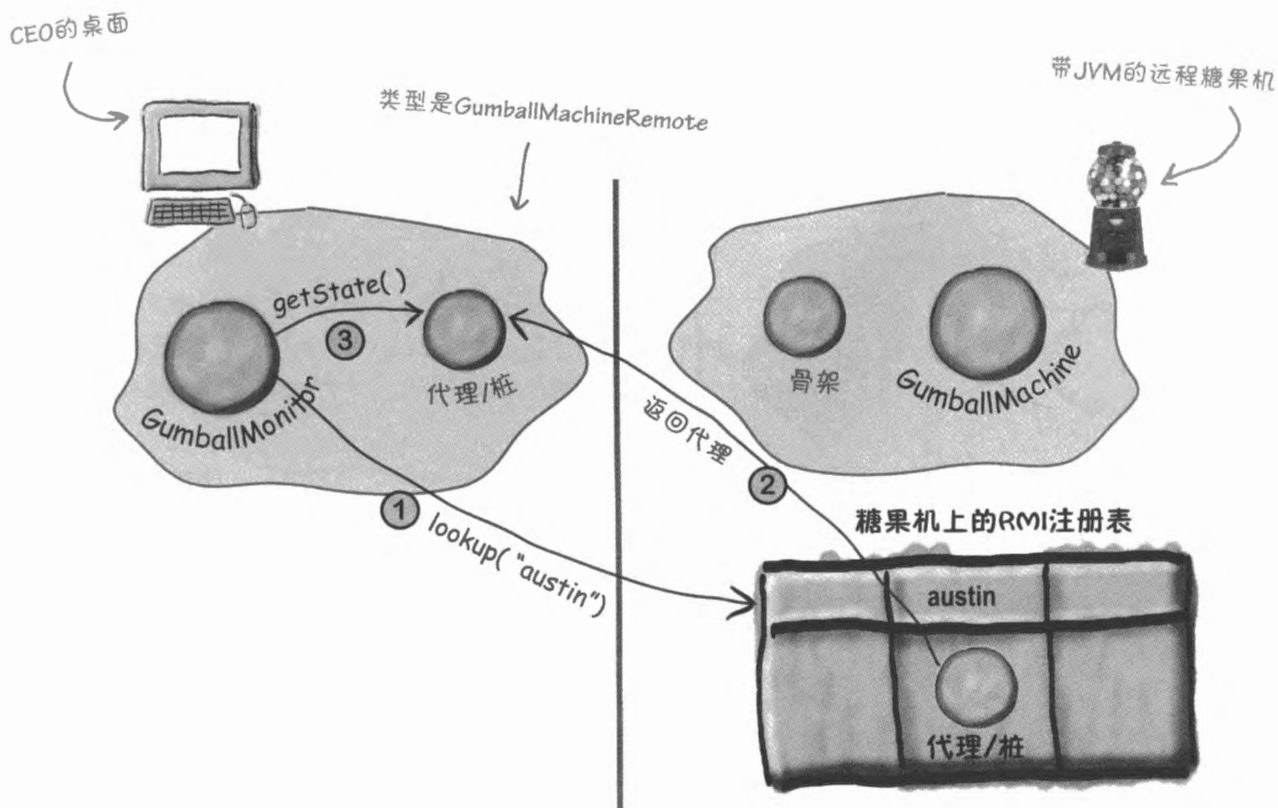
幕后花絮



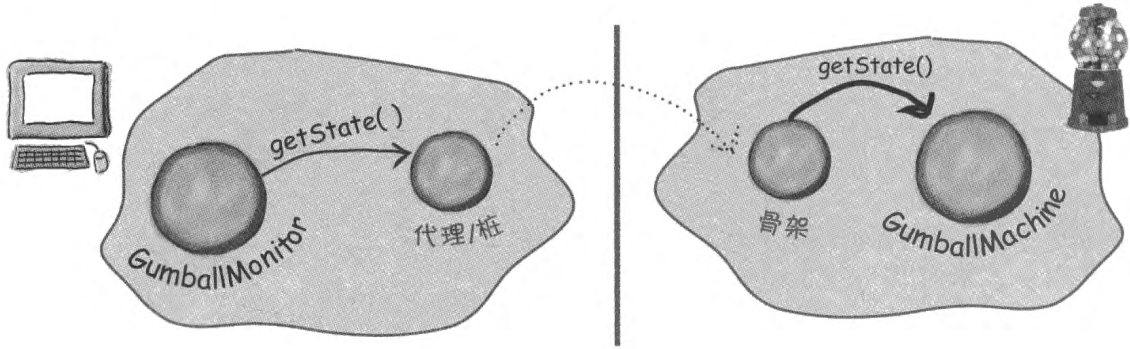
这一切运作得很好，但我要确认自己是怎么做到的。



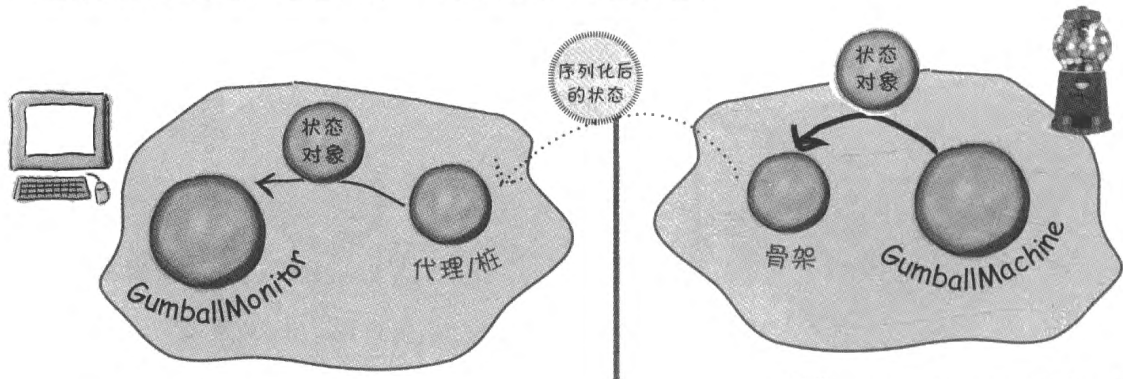
- 1 CEO运行监视器，先取得远程糖果机的代理，然后调用每个代理的 `getState()`（以及 `getCount()` 和 `getLocation()`）。



- 2 代理上的getState()被调用，此调用被转发到远程服务。骨架接收到请求，然后转发给糖果机。



- 3 糖果机将状态返回给骨架，骨架将状态序列化，通过网络传回给代理，代理将其反序列化，把它当作一个对象返回给监视器。



监视器根本没有改变，除了它知道可能会遭遇远程异常。它也使用 `GumballMachineRemote` 接口，而不是具体的实现。

同样，`GumballMachine` 实现了另一个接口，而且可能在构造器中抛出远程异常。除此之外，代码不做改变。

我们也有一些代码负责用RMI注册表来注册和定位桩，但是无论如何，如果我们h需要编写一些在互联网上工作的东西，我们就需要这样一些定位器服务。

定义代理模式

这一章已经有好多页了，因为我们花了很多时间在解释远程代理。尽管如此，你还是会看到，代理模式的定义和类图其实相当直白。注意，远程代理是一般代理模式的一种实现，其实这个模式的变体相当多，我们稍后会谈到这些变体。

现在，我们来了解通用模式的细节。

下面是代理模式的定义：

代理模式为另一个对象提供一个替身或占位符来控制对这个对象的访问。

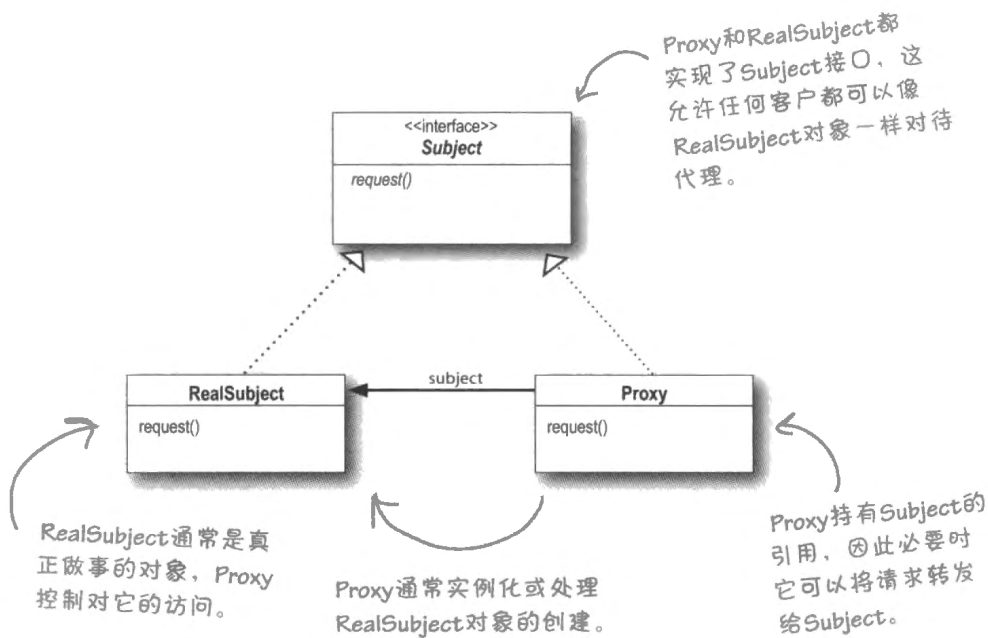
我们已经看到代理模式如何为另一个对象提供替身或占位符，我们也把代理描述成另一个对象的“代表”。

但是代理控制访问怎么解释？这听起来有一点奇怪。别担心，在糖果机的例子中，就可以认为代理控制远程对象的访问。代理之所以需要控制访问，是因为我们的客户（监视器）不知道如何和远程对象沟通。在某种意义上，远程代理控制访问，以便为我们处理网络细节。正如刚刚讨论的，代理模式有许多变体，这些变体通常和“控制访问”的做法有关。稍后会更详细地讨论这个问题，目前我们先看看几种代理控制访问的方式：

- 我们已经知道的，远程代理控制对远程对象的访问。
- 虚拟代理控制对创建开销大的资源的访问。
- 保护代理基于权限控制对资源的访问。

既然你对通用模式已经有了概要的认识，来看看类图……

使用代理模式创建代表对象。代表对象控制对另一个对象的访问，被代表的对象可以是远程的对象、创建开销大的对象或需要安全控制的对象。



让我们详细看这张图……

首先，我们有一个Subject，它为RealSubject和Proxy提供了接口。因为实现了和RealSubject一样的接口，Proxy可以在RealSubject出现的任何地方取代它。

RealSubject是真正做事的对象，它是被Proxy代表和控制访问的对象。

Proxy持有RealSubject的引用。在某些例子中，Proxy还会负责创建和销毁RealSubject。客户通过Proxy和RealSubject交互。因为Proxy和RealSubject实现相同的接口（Subject），因此任何用到Subject的地方，都可以用Proxy取而代之。Proxy也控制对RealSubject的访问，在某些情况下这样的控制是需要的，比如Subject运行在远程机器上，Subject创建开销大或者Subject的访问需要被保护。

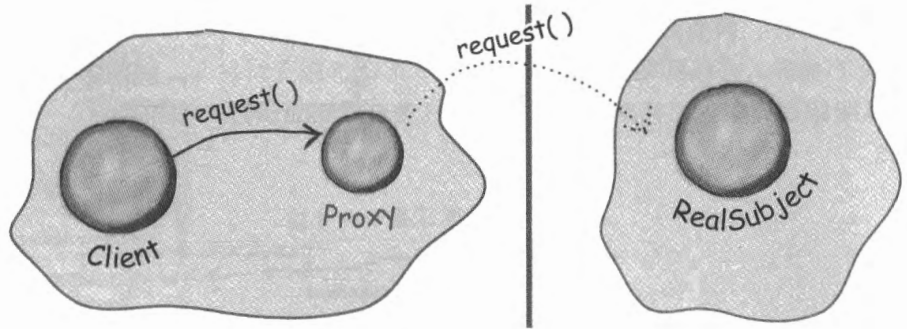
现在你已经了解了通用的模式，我们来看看，除了远程代理之外，还有哪些使用代理的方式……

轮到虚拟代理了 (Virtual Proxy)

好啦，你已经看过代理模式的定义，也看了一个特定的例子（远程代理）。现在来看看一种不同类型的代理，虚拟代理。你将发现，代理模式可以以很多形式显现，但大体上都符合通用的代理设计。为什么有如此多的形式？因为代理模式可以被用在许多不同的例子中。让我们现在看看虚拟代理并把它和远程代理比较：

远程代理

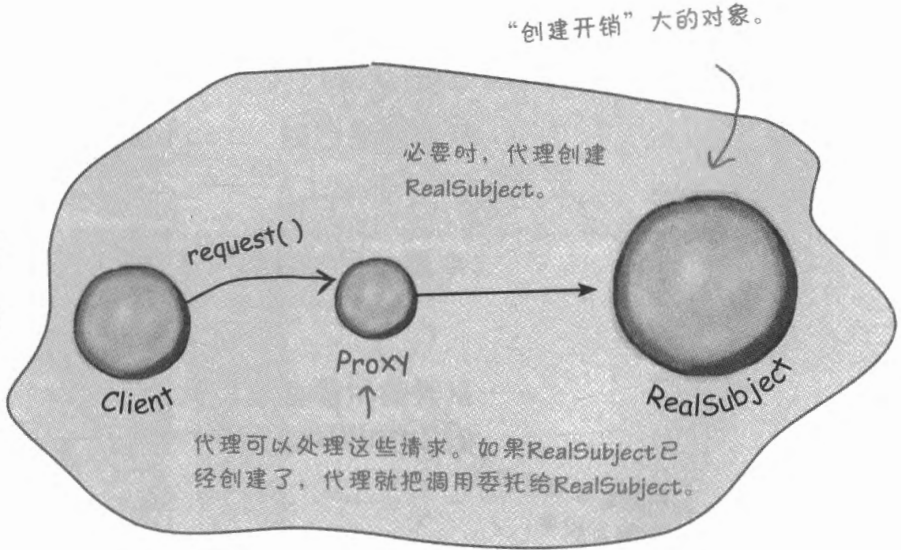
远程代理作为另一个JVM上对象的本地代表。调用代理的方法，会通过网络转发到远程来调用，结果返回给代理，再由代理将结果转给客户。



我们已经很熟悉这个图了……

虚拟代理

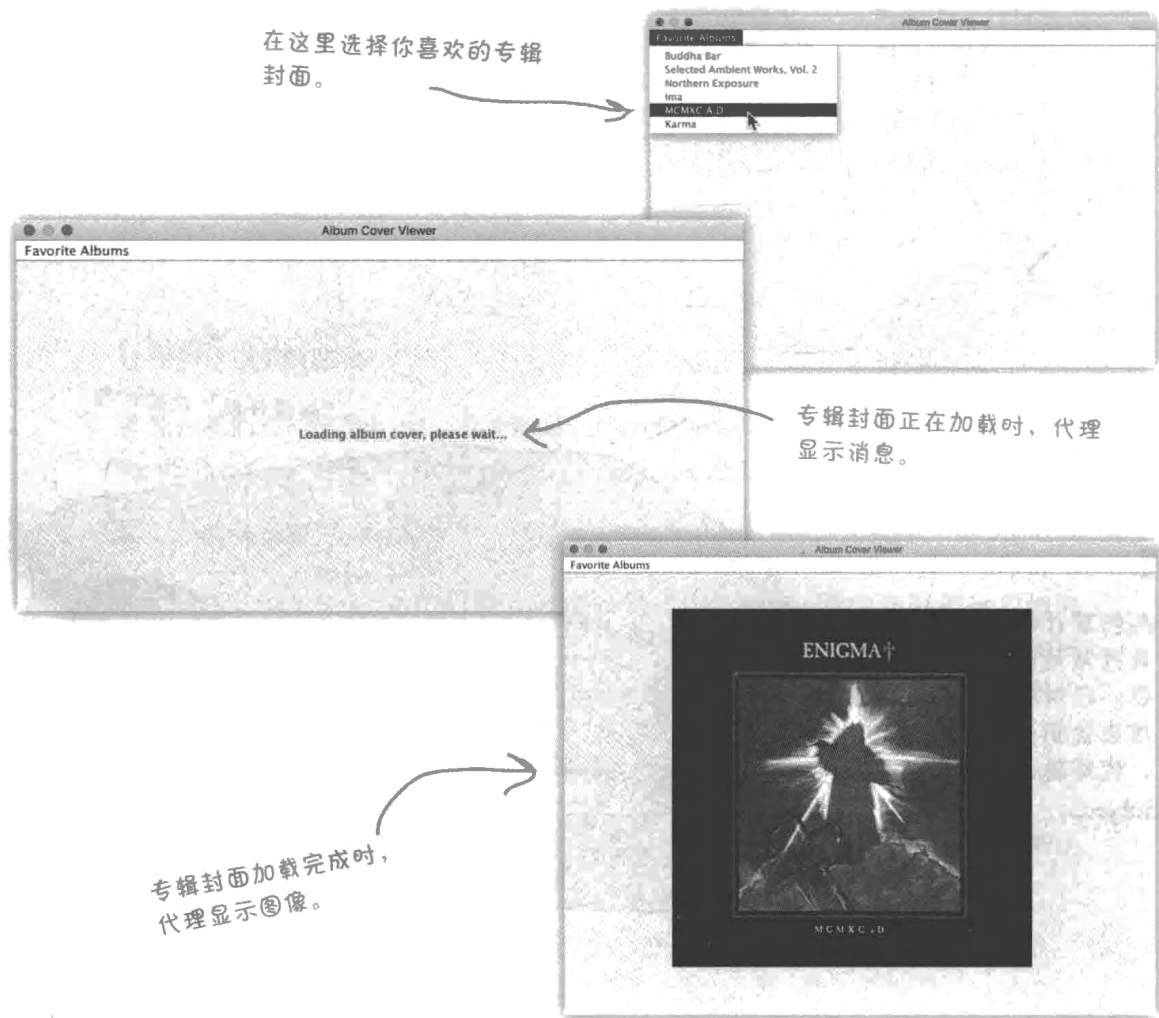
虚拟代理作为创建开销大的对象的代表。虚拟代理经常延迟对象的创建，直到真正需要。在对象创建前和创建时，虚拟代理也会扮演对象的替身。对象创建后，代理就会将请求直接委托给RealSubject。



显示音乐专辑封面

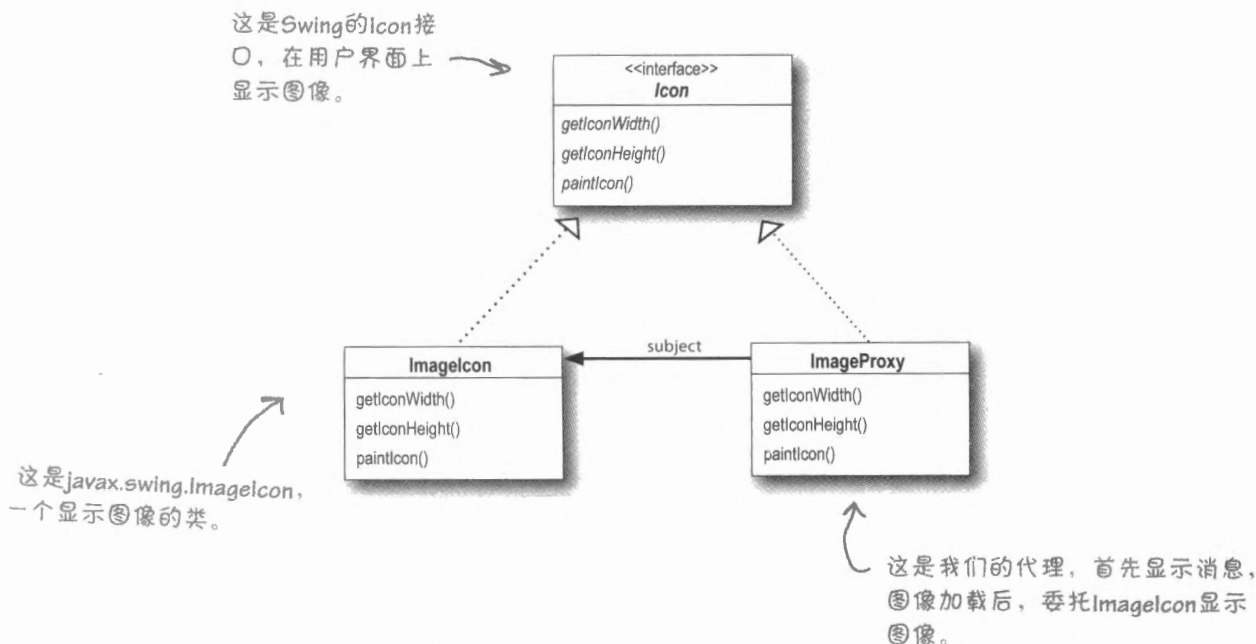
假设我们要编写一个应用，用来展现你喜爱的音乐专辑封面。你可以建立一个专辑标题菜单，然后从Amazon.com等网站的在线服务中取得音乐专辑封面的图。如果你使用Swing，可以创建一个Icon，让它从网络取得图像。唯一的问题是，限于网络负载和连接带宽，取得一张专辑封面可能需要一些时间，因此在等待图像加载时，你的应用应该显示一些东西。我们也不希望在等待图像时整个应用被挂起。一旦图像被加载，刚才显示的东西应该消失，图像显示出来。

想做到这样，简易的方式就是通过虚拟代理。虚拟代理可以扮演图像，管理背后的加载，并在网络加载完成前显示“正在加载专辑封面，请稍候……”，一旦图像加载完成，代理就把显示的责任委托给Icon。



设计音乐专辑封面虚拟代理

在开始编写音乐专辑封面浏览器代码之前，让我们看一下类图。这张类图和远程代理的图很类似，但是这里的代理是用来隐藏创建开销大的对象（因为我们需要通过网络取得图像数据），而不是生存在网络其他地方的对象。

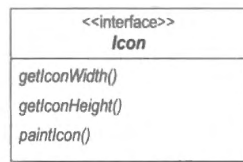


ImageProxy是如何工作的：

- 1 ImageProxy首先创建一个ImageIcon，然后开始从网络URL上加载图像。
- 2 在加载的过程中，ImageProxy显示“专辑封面加载中，请稍候……”。
- 3 当图像加载完毕，ImageProxy把所有方法调用委托给ImageIcon，这些方法包括paintIcon()、getWidth()和getHeight()。
- 4 如果用户请求新的图像，我们就创建新的代理，又开始这样的过程。

编写Image Proxy

ImageProxy实现
Icon接口。



这个imageIcon是真正的图像，我们希望
在加载后最终显示的图像。

我们将图像的URL传入构造器，这是我
们希望加载后显示的图像。

等到imageIcon加载完毕，
我们返回缺省的宽和高，
并转给imageIcon处理。

imageIcon被两个不同的线程使用，
在使变量volatile的同时（以保护
读），我们使用了同步setter（以
保护写）。

有趣的地方在这里。这里的代码会在
屏幕上画出一个图像（通过委托给
imageIcon）。然而，如果我们没有
完整创建的imageIcon，就创建一个。
下一页会看得更清楚……

```
class ImageProxy implements Icon {
    volatile ImageIcon imageIcon;
    final URL imageURL;
    Thread retrievalThread;
    boolean retrieving = false;

    public ImageProxy(URL url) { imageURL = url; }
    public int getIconWidth() {
        if (imageIcon != null) {
            return imageIcon.getIconWidth();
        } else {
            return 800;
        }
    }
    public int getIconHeight() {
        if (imageIcon != null) {
            return imageIcon.getIconHeight();
        } else {
            return 600;
        }
    }

    synchronized void setImageIcon(ImageIcon imageIcon) {
        this.imageIcon = imageIcon;
    }

    public void paintIcon(final Component c, Graphics g, int x, int y) {
        if (imageIcon != null) {
            imageIcon.paintIcon(c, g, x, y);
        } else {
            g.drawString("Loading album cover, please wait...", x+300, y+190);
            if (!retrieving) {
                retrieving = true;

                retrievalThread = new Thread(new Runnable() {
                    public void run() {
                        try {
                            setImageIcon(new ImageIcon(imageURL, "Album Cover"));
                            c.repaint();
                        } catch (Exception e) {
                            e.printStackTrace();
                        }
                    }
                });
                retrievalThread.start();
            }
        }
    }
}
```



Code up Close

当需要在屏幕上绘制图像时，调用这个方法。

```

public void paintIcon(final Component c, Graphics g, int x, int y) {
    if (imageIcon != null) {

        imageIcon.paintIcon(c, g, x, y);

    } else {

        g.drawString("Loading album cover, please wait...", x+300, y+190);
        if (!retrieving) {

            retrieving = true;
            retrievalThread = new Thread(new Runnable() {
                public void run() {
                    try {
                        setImageIcon(new ImageIcon(imageURL, "Album Cover"));
                        c.repaint();
                    } catch (Exception e) {
                        e.printStackTrace();
                    }
                }
            });

            retrievalThread.start();
        }
    }
}

```

如果已经有图像，就告诉它画出自己。

否则显示“下载中”消息。

我们在这里加载真正的icon图像。注意，图像加载和ImageIcon是同步的，也就是说，直到图像已经加载，ImageIcon构造器才会返回。这样屏幕无法更新或显示消息，因此要把加载变成异步的。下一页会详细说明……



Code Way Up Close

如果我们还没有尝试获取图像……

……那么就开始获取图像（不用担心，只有一个线程会调用paint，因此是线程安全的）。

```
if (!retrieving) {
    retrieving = true;
```

我们不想挂起整个用户界面，因此用另一个线程获取图像。

```
    retrievalThread = new Thread(new Runnable() {
        public void run() {
            try {
                setImageIcon(new ImageIcon(imageURL, "Album Cover"));
                c.repaint();
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    });
    retrievalThread.start();
}
```

有了图像之后，我们告诉Swing需要重绘。

在线程中，我们实例化此Icon对象。它的构造器在图像加载后才会返回。

因此，下一次屏幕绘制是在实例化ImageIcon之后，paintIcon()方法绘制真正的图像，而不是正在下载的消息。



设计谜题

ImageProxy类看起来有两个状态，由条件语句控制。你能否用另一个模式清理这样的代码？你要如何重新设计ImageProxy？

```
class ImageProxy implements Icon {
```

```
    // 实例变量构造器在这里
```

```
    public int getIconWidth() {
```

```
        if (imageIcon != null) {
```

```
            return imageIcon.getIconWidth();
```

```
        } else {
```

```
            return 800;
```

```
        }
```

```
    }
```

```
    public int getIconHeight() {
```

```
        if (imageIcon != null) {
```

```
            return imageIcon.getIconHeight();
```

```
        } else {
```

```
            return 600;
```

```
        }
```

```
    }
```

```
    public void paintIcon(final Component c, Graphics g, int x, int y) {
```

```
        if (imageIcon != null) {
```

```
            imageIcon.paintIcon(c, g, x, y);
```

```
        } else {
```

```
            g.drawString("Loading album cover, please wait...", x+300, y+190);
```

```
            // 更多的代码在这里
```

```
        }
```

```
    }
```

```
}
```

两个状态

两个状态

两个状态

测试音乐专辑封面浏览器



现在就试试这个崭新的虚拟代理。在后台我们已经烘烤好了一个新的 ImageProxyTestDrive，用来设置窗口、创建框架、安装菜单和创建我们的代理。我们不在这里研究这些代码的细节，虚拟代理的源代码列在本章最后，你可以随时去研究。

下面是部分测试代码：

```
public class ImageProxyTestDrive {
    ImageComponent imageComponent;
    public static void main (String[] args) throws Exception {
        ImageProxyTestDrive testDrive = new ImageProxyTestDrive();
    }
```

```
public ImageProxyTestDrive() throws Exception {
```

```
    // 设置框架和菜单
```

```
    Icon icon = new ImageProxy(initialURL);
    imageComponent = new ImageComponent(icon);
    frame.getContentPane().add(imageComponent);
```

```
}
```

```
}
```

在这里，我们创建一个图像代理，并指定初始URL。无论何时你从音乐专辑菜单中选择，就会得到一个新的图像代理。

接下来，我们把代理包装进一个组件，以便可以加进框架。组件会处理代理的宽度、高度之类的细节。

最后我们把代理加进框架中，这样它可以被显示。

现在执行测试程序：

File Edit Window Help JustSomeOfTheAlbumsThatGotUsThroughThisBook

```
% java ImageProxyTestDrive
```

运行ImageProxyTestDrive，应该看到这样的窗口。

要测试这些……

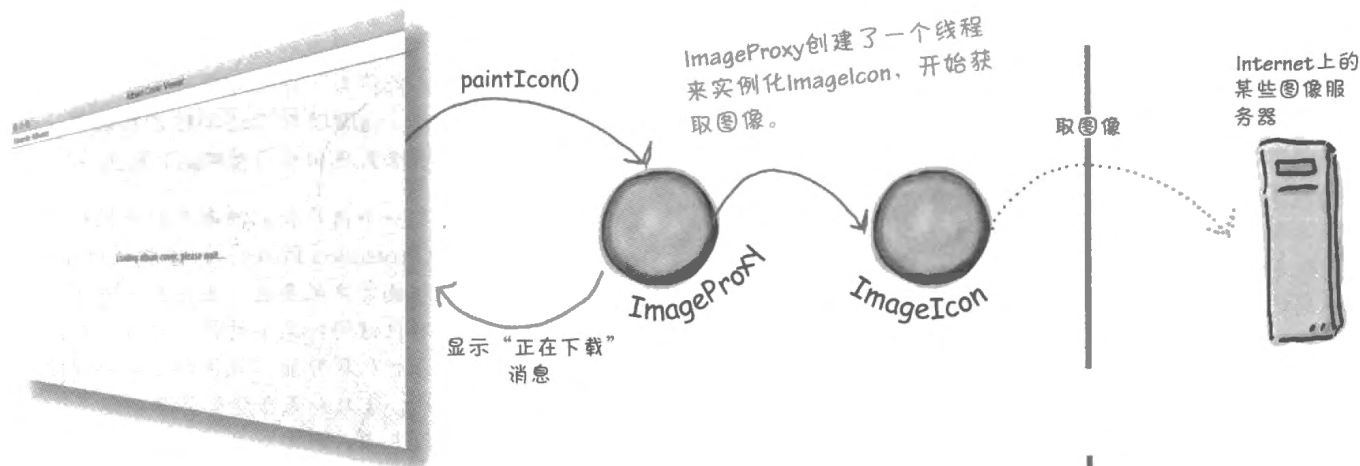
- 1 用菜单加载不同的专辑封面，然后看着代理显示“正在加载”，直到出现真正的图像。
- 2 画面出现“正在加载”消息时，缩放窗口大小，注意，代理会在不挂起Swing窗口的情况下处理加载。
- 3 把你最喜欢的音乐专辑加进ImageProxyTestDrive。



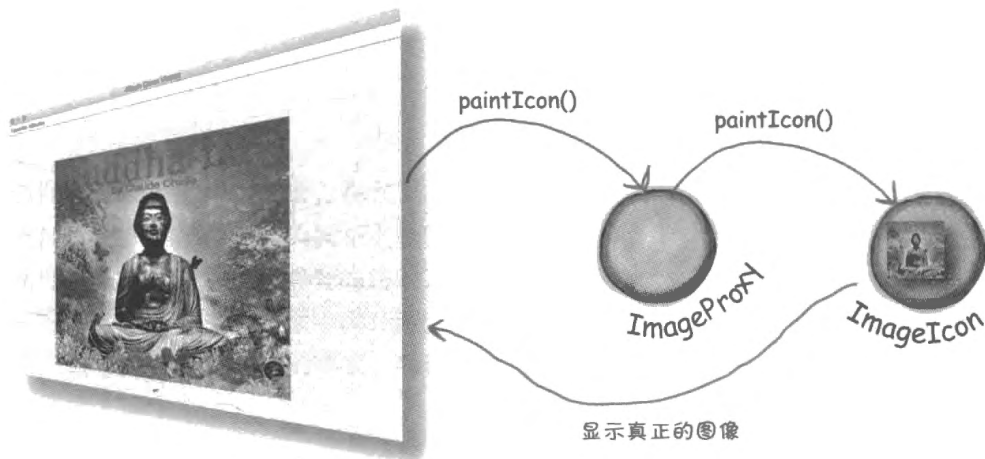
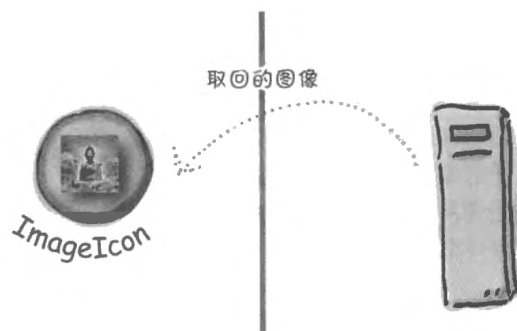
我们做了什么？

- 1 我们创建了一个ImageProxy类用来显示。paintIcon()方法会被调用，而ImageProxy会引发线程获取图像，并创建ImageIcon。

幕后
花絮



- 2 在某个时间点图像被返回，ImageIcon完整实例化。
- 3 ImageIcon被创建后，下次调用到paintIcon()时，代理就委托给ImageIcon。



没有蠢问题

问：对我来说，远程服务器和虚拟服务器差异如此之大，它们真的是一个模式吗？

答：在真实的世界中，代理模式有许多变体，这些变体都有共通点：都会将客户对主题（Subject）施加的方法调用拦截下来。这种间接的级别让我们可以做许多事，包括将请求分发到远程主题，为创建开销大的对象提供代表，或者你将看到的，提供某些级别的保护，这种保护能决定哪些客户应该调用哪些方法。这还只是个开端，其实一般的代理模式还可以有许多形式的应用，本章最后会略提其中几种变体。

问：ImageProxy在我看来好像是装饰者。我的意思是，基本上都是把对象用另一个对象包起来，然后把调用委托给ImageIcon。我这样说有什么漏洞吗？

答：有时候两者看起来很像，但是它们的目的是不一样的。装饰者为对象添加行为，而代理控制对象的访问。你可能会问，“显示正在加载消息难道不是在添加行为吗？”从某方面来说，算是，但是，更重要的，ImageProxy控制ImageIcon的访问。如何控制？试想：代理将客户从

ImageIcon解耦，如果它们之间是耦合的，客户就必须等待，直到每幅图像都被取回，然后才能绘制在界面上。代理控制ImageIcon的访问，以便在图像完全创建之前，提供屏幕上的代表，一旦ImageIcon被创建，代理就允许访问ImageIcon。

问：我要如何让客户使用代理，而不是真正的对象？

答：好问题。一个常用的技巧是提供一个工厂，实例化并返回主题。因为这是在工厂方法内发生的，我们可以用代理包装主题再返回，而客户不知道，或者不在意它使用的是代理而不是真东西。

问：我注意到，在ImageProxy的例子中，你总是创建新的ImageIcon取得图像，即使图像已经被取回来过。能不能在ImageProxy中实现一些类似缓存的东西？

答：你说的是虚拟代理的一种特殊形式，叫作缓存代理（Caching Proxy）。缓存代理会维护之前创建的对像的缓存，当收到请求时，如果可能的话返回缓存对象。本章最后会介绍代理模式的数种变体。

问：我已经知道装饰者和代理的关系了，但是适配器呢？代理和适配器也很类似。

答：代理和适配器都是挡在其他对象的前面，并将请求转发给它们。记住，适配器改变适配对象的接口，而代理实现相同的接口。

有一个额外相似性牵涉到保护代理（Protection Proxy）。保护代理可以根据客户的角色，决定允许或不允许客户访问对象中的特定方法，因此保护代理可能只提供部分的接口给客户，这就和某些适配器很相像了。再过几页，我们会讨论到保护代理。

炉边谈话



今夜话题：代理和装饰者的意图

代理：

你好，装饰者。我猜之所以你在这里，是因为又是人们会把我们搞混？

我抄袭你的想法？拜托，我控制对象的访问，你只是装饰对象。我的工作比你重要多了。这一点都不好笑。

好吧！或许也不是完全无足轻重……但是我还是不知道你为什么认为我在抄袭你，我是代表主题，不是装饰它们。

我想你还是没搞懂，装饰者。我代表主题，不只是添加行为。客户用我作为真正主题的替身，因为我可以保护对象避免不想要的访问，也可以避免在等待加载大对象的过程中GUI会挂起，或者隐藏主题在远程机器运行的事实。我的意图和你的差别很大！

装饰者：

我认为人们把我们搞混的原因是你到处招摇撞骗，说你是一个全然不同的模式。事实上，你只不过是乔装过的装饰者。我真的觉得你不应该抄袭我的想法。

“只是”装饰？你认为装饰无足轻重？我告诉你这个家伙，我为对象添加行为，这是对象最重要的事：它们做什么！

你可以把它称为“代表”，但如果看着像鸭子，走着像鸭子……我是说，看看你的虚拟代理，只是加入行为的另一种方式，在创建开销大的对象时做一些事情，还有你的远程代理，就是一种和远程对象沟通的方法，这样客户就不用操心了。全都是关于行为，就像我所说的。

你爱怎么说都行。我实现和所包装对象相同的接口，你不也是吗！

代理：

好，我来评点一下你说的话。你包装了一个对象，有时候我们非正式地说，代理包装了它的主题，这样说其实并不精确。

想想远程代理……我包装了什么对象？我所代表和控制访问的对象是在另一台机器上！你做得到吗？

当然有了，以虚拟代理来说……想想音乐专辑浏览器的例子。当客户第一次用我作为代理时，主题甚至还不存在！你说我包装谁？

我不知道装饰者会这么笨！当然我有时候会创建对象，不然你以为虚拟代理怎么取得主题的！好了，你刚刚指出了我们之间的一个大差异：我们都知道装饰者只能装饰点缀，你们从来不会实例化任何东西。

嘿，经过这次谈话，我确信你是个笨蛋代理。

你很少看到代理将一个主题包装多次，事实上，如果你真的把某些对象包装十次，你最好回去重新检查你的设计。

装饰者：

是吗？为什么不是？

OK，但我们都知道，远程代理是个奇葩，我不相信你可以找出第二个例子。

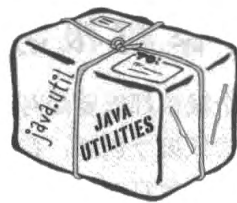
哼哼，我猜想接下来你会说对象其实是你创建的呢。

是吗？实例化一下这个。

笨蛋代理？我倒想看看你有没有能耐把一个对象用十个装饰者递归包装，还能直起你的头。

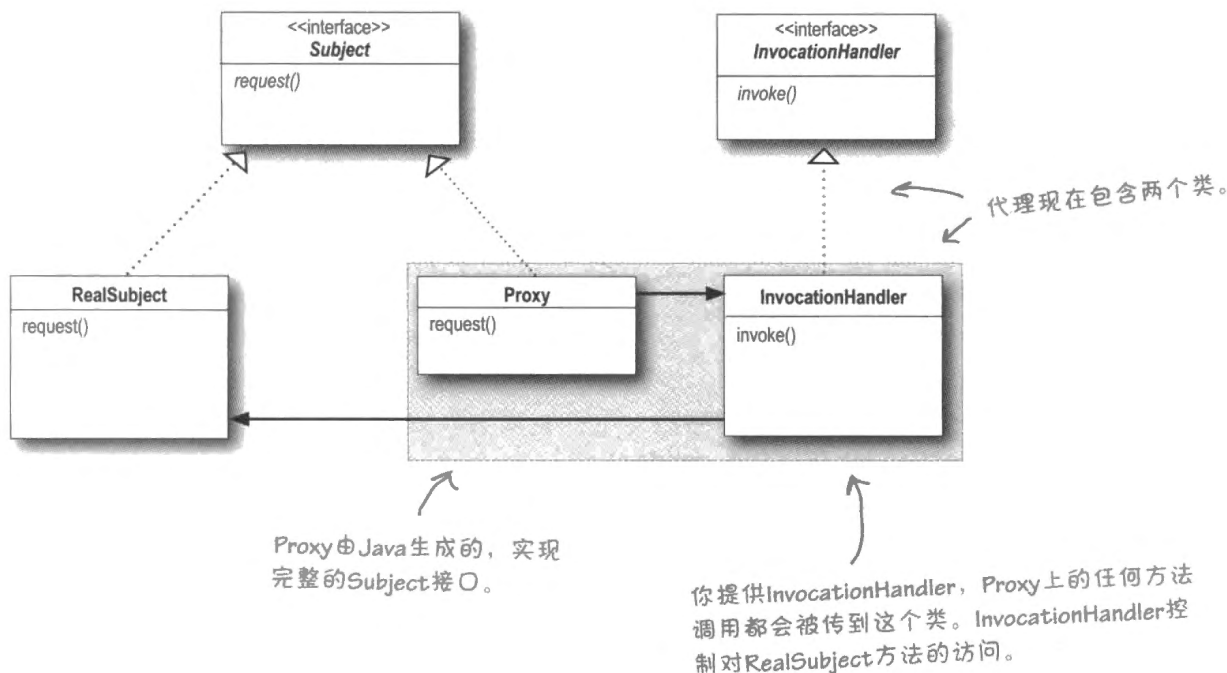
果然像代理，装得跟真的一样，事实上，你们只是挡在做实事的对象的前面。我真替你们感到悲哀。

使用Java API的代理来创建一个保护代理



Java在java.lang.reflect包中有自己的代理支持，利用这个包你可以动态地创建一个代理类，实现一个或多个接口，将方法的调用转发到你所指定的类。因为实际的代理类是在运行时创建的，我们把这个Java技术称为：动态代理。

我们打算用Java的动态代理来创建我们的下一个代理实现（保护代理）。但在这之前，先快速看一眼类图，了解动态代理涉及到哪些类。就像真实世界的大多数事物，它和代理模式的经典定义稍有差别。



因为Java为你创建了Proxy类，所以你需要有办法来告诉Proxy类要做什么。你不能像以前一样把代码放在Proxy类中，因为Proxy不是你直接实现的。因此，如果这样的代码不能放在Proxy类中，放在哪里呢？放在InvocationHandler中。InvocationHandler的工作是响应代理的任何方法调用。把InvocationHandler想成是代理收到方法调用后，请求做实际工作的对象。

OK，接下来看看如何使用动态代理……

对象村极客配对

每个城镇都需要配对服务，不是吗？你负责帮对象村实现约会服务系统。你尝试搞个创新，在服务中加入“极客评分”特性，参加者可以给对方的极客程度打分。你希望这可以让顾客积极参与并找到可能的对象，这也会让事情更有趣。

你的服务系统牵涉到一个Person 接口，该接口允许设置并取得一个人的信息：



极客配对

这是一个接口，我们稍后
就会实现它……

```
public interface Person {
```

```
    String getName();  
    String getGender();  
    String getInterests();  
    int getGeekRating();
```

```
    void setName(String name);  
    void setGender(String gender);  
    void setInterests(String interests);  
    void setGeekRating(int rating);
```

```
}
```

这里可以取得人的名字、性别、
兴趣和极客评分（1~10）。

通过各自的方法调用，
我们也可以设置同样的
信息。

setGeekRating() 需要一个整数
作为参数，并将它添加到这个
人的运行平均值。

现在来看看实现……

Person实现

```
public class PersonImpl implements Person {
```

```
    String name;
```

```
    String gender;
```

```
    String interests;
```

```
    int rating;
```

```
    int ratingCount = 0;
```

```
    public String getName() {
```

```
        return name;
```

```
    }
```

```
    public String getGender() {
```

```
        return gender;
```

```
    }
```

```
    public String getInterests() {
```

```
        return interests;
```

```
    }
```

```
    public int getGeekRating() {
```

```
        if (ratingCount == 0) return 0;
```

```
        return (rating/ratingCount);
```

```
    }
```

```
    public void setName(String name) {
```

```
        this.name = name;
```

```
    }
```

```
    public void setGender(String gender) {
```

```
        this.gender = gender;
```

```
    }
```

```
    public void setInterests(String interests) {
```

```
        this.interests = interests;
```

```
    }
```

```
    public void setGeekRating(int rating) {
```

```
        this.rating += rating;
```

```
        ratingCount++;
```

```
    }
```

PersonImpl实现Person接口。

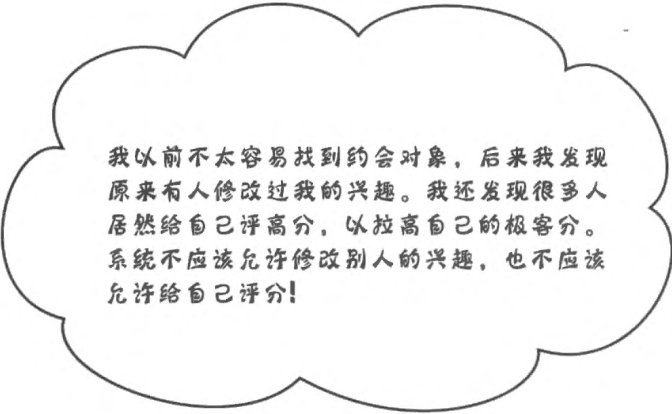
实例变量。

所有Getter方法。它们各自返回相应的实例变量……

除了getGeekRating(),它通过rating除以ratingCount来计算rating的平均值。

这是所有的Setter方法,设定相应的实例变量。

最后, setGeekRating()方法增加总的ratingCount,并将rating加到运行时总计。



虽然我们怀疑Elroy找不到约会对象是因为其他的因素，但是他说的没错，不应该给自己投票或修改别人的数据。根据我们定义Person的方式，任何客户都可以调用任何方法。

这是一个我们可以使用保护代理的绝佳例子。什么是保护代理？这是一种代理，基于访问权限控制对象的访问。例如，如果你有一个雇员对象，保护代理允许雇员调用对象上的某些方法，经理还可以多调用一些其他的方法（例如setSalary()），而人力资源处的雇员可以调用对象上的所有方法。

在我们的约会服务中，我们要确保顾客可以设置自己的信息，同时防止他人更改这些信息。极客值评分则相反，你不能修改自己的评分，但是其他顾客可以设置你的评分。我们在Person中已经有许多getter方法，每个方法的返回信息都是公开的，任何顾客都可以调用它们。



五分钟短剧：保护主题

互联网泡沫已经成为遥远的记忆。在那些日子里，如果你要找一个更好更高薪的工作，街对面就能找到。甚至软件开发人员的经纪人也赶上这股风潮……



全貌：为Person创建动态代理

我们有一些问题要修正：顾客不应该改变自己的极客值评分，也不能改变其他顾客的个人信息。要修正这些问题，我们要创建两个代理：一个访问你自己的Person对象，另一个访问另一顾客的Person对象。这样，代理就可以控制在每一种情况下允许哪一种请求。

要创建这种代理，我们必须使用几页前提到的Java API动态代理。Java会为我们创建两个代理，我们只须提供处理器，当代理上的方法被调用时，它知道怎么做。

步骤一：

创建两个 **InvocationHandlers**。

InvocationHandler实现代理的行为，正如你将看到的，Java负责创建真实代理类和对象。我们只须提供在方法调用发生时知道做什么的处理器。

步骤二：

写代码创建动态代理。

我们需要写一些代码产生代理类，并实例化它。等一下就会看到这些代码。

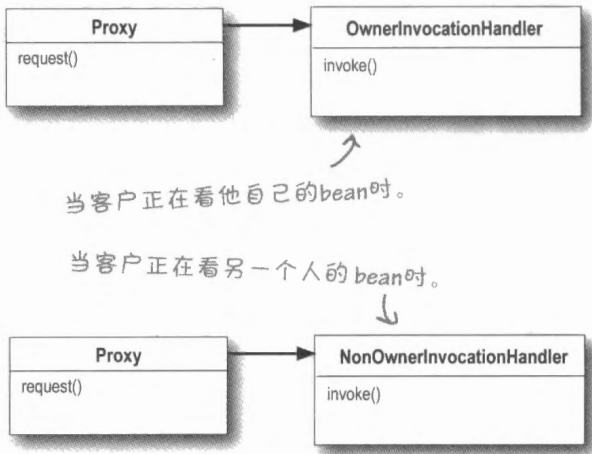
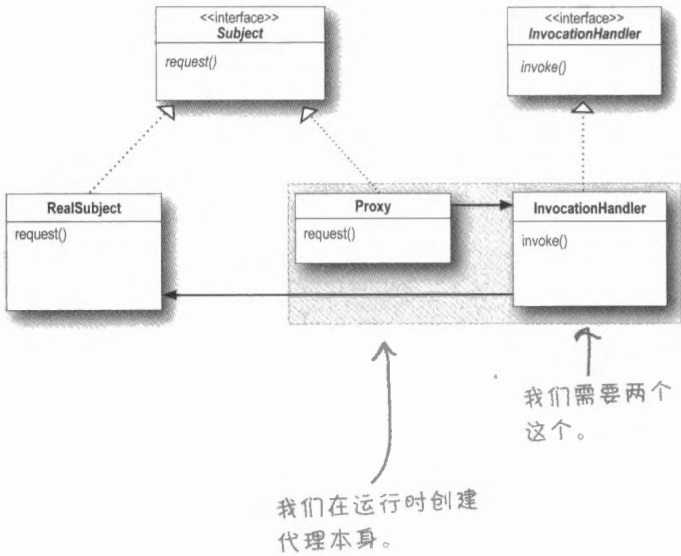
步骤三：

用适当的代理包装任何Person对象。

当我们需要使用Person对象时，要么是顾客自己（这种情况我们叫它“所有者”）的对象，要么是顾客正在检查的服务的另一个使用者（这种情况我们叫它“非所有者”）的对象。

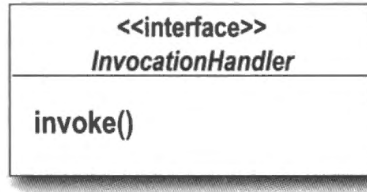
不管是哪一种情况，我们都为Person创建合适的代理。

还记得几页之前的这张图吧？



步骤一：创建调用处理器

我们知道需要写两个调用处理器，其中一个给所有者，另一个给非所有者。但什么是调用处理器？你可以这么想，当代理的方法被调用，代理就会把此调用转发给调用处理器，但是这并不是通过调用调用处理器的相应方法做到的。那么，是如何做到的？看看InvocationHandler的接口：



只有一个方法`invoke()`，不管代理被调用的是什么方法，处理器被调用的一定是`invoke()`方法。看看这是如何运作的：

- ① 假设proxy上的`setGeekRating()`方法被调用。

`proxy.setGeekRating(9);`

- ② proxy会接着调用InvocationHandler上的`invoke()`方法。

`invoke(Object proxy, Method method, Object[] args)`

Method类是reflection API的一部分，利用它的`getName()`方法，就知道proxy被调用的方法是什么。

- ③ 处理器决定要如何处理该请求，可能会转发给RealSubject。处理器如何决定？一会就知道了。

如何调用RealSubject上的方法。

`return method.invoke(person, args);`

在这里，我们调用原本proxy被调用的方法。这个对象在调用里传给我们。

这时我们才在RealSubject上调用它。

……使用原本的参量。

继续创建InvocationHandler……

当proxy调用invoke()时，怎么知道该做什么？通常，你会先检查该方法是否来自proxy，并基于方法的名称（可能还有参量）做决定。现在来实现OwnerInvocationHandler，以了解运作机制：

InvocationHandler是java.lang.reflect包的一部分，因此我们需要import它。

```
import java.lang.reflect.*;
```

所有调用处理器都实现InvocationHandler接口。

```
public class OwnerInvocationHandler implements InvocationHandler {
```

我们把RealSubject传入构造器，并保持它的引用。

```
    Person person;
```

```
    public OwnerInvocationHandler(Person person) {
        this.person = person;
    }
```

在这里，每次proxy上的方法被调用，invoke()方法被调用。

```
    public Object invoke(Object proxy, Method method, Object[] args)
        throws IllegalAccessException {
```

```
        try {
            if (method.getName().startsWith("get")) {
                return method.invoke(person, args);
            } else if (method.getName().equals("setGeekRating")) {
                throw new IllegalAccessException();
            } else if (method.getName().startsWith("set")) {
                return method.invoke(person, args);
            }
        } catch (InvocationTargetException e) {
            e.printStackTrace();
        }
        return null;
    }
```

如果方法是一个getter，就在真正主题上调用它。

否则，如果方法是setGeekRating()，就抛出IllegalAccessException表示不允许。

如果真正主题丢出异常，就会执行这里。

因为我们是所有者，任何其他set方法都可以，我们就在真正主题上调用它。

如果调用其他的方法，一律返回null，不存侥幸。



NonOwnerInvocationHandler的运作方式就像OwnerInvocationHandler，除了这些区别：它允许调用setGeekRating()而且不允许调用任何其他set方法。请自行写出这个处理器：

步骤二：创建Proxy类并实例化Proxy对象

现在，只剩下创建动态Proxy类，并实例化Proxy对象。让我们从编写一个方法开始。该方法以Person对象为参数，并知道如何为其创建所有者代理。也就是说，我们要创建一种代理，将它的方法调用转发给OwnerInvocationHandler。代码如下：

```

    这个方法需要一个Person对象（真正的主题）作为
    参数，然后为其返回代理，因为代理和主题有相
    同的接口，因此我们返回一个Person。
    ↓
    Person getOwnerProxy(Person person) {

        return (Person) Proxy.newProxyInstance(
            person.getClass().getClassLoader(),
            person.getClass().getInterfaces(),
            new OwnerInvocationHandler(person));
    }
  
```

这份代码创建代理，可能有点丑，查看时要小心。

我们用Proxy类上的静态newProxyInstance()方法创建代理。

把类载入器传给它……

代理需要实现的接口集。

调用处理器，在这里是OwnerInvocationHandler。

将真正的主题传入调用处理器的构造器。如果你往回看两页，就会发现这正是处理器能够访问真实主题的原因。



削尖你的铅笔

虽然有一点复杂，但是创建动态代理所需要的代码其实不多。请你编写getNonOwnerProxy()，该方法为NonOwnerInvocationHandler返回代理：

更进一步，请编写getProxy()方法，该方法参数是处理器和person，返回使用此处理器的代理。

测试配对服务

现在就来试试配对服务，看看它如何基于所用代理控制对 setter 方法的访问。

```
public class MatchMakingTestDrive {
    // 这里有实例变量

    public static void main(String[] args) {
        MatchMakingTestDrive test = new MatchMakingTestDrive();
        test.drive();
    }

    public MatchMakingTestDrive() {
        initializeDatabase();
    }

    public void drive() {
        Person joe = getPersonFromDatabase("Joe Javabeen");
        Person ownerProxy = getOwnerProxy(joe);
        System.out.println("Name is " + ownerProxy.getName());
        ownerProxy.setInterests("bowling, Go");
        System.out.println("Interests set from owner proxy");
        try {
            ownerProxy.setGeekRating(10);
        } catch (Exception e) {
            System.out.println("Can't set rating from owner proxy");
        }
        System.out.println("Rating is " + ownerProxy.getGeekRating());

        Person nonOwnerProxy = getNonOwnerProxy(joe);
        System.out.println("Name is " + nonOwnerProxy.getName());
        nonOwnerProxy.setInterests("bowling, Go");
        } catch (Exception e) {
            System.out.println("Can't set interests from non owner proxy");
        }
        nonOwnerProxy.setGeekRating(3);
        System.out.println("Rating set from non owner proxy");
        System.out.println("Rating is " + nonOwnerProxy.getGeekRating());
    }

    // 这里还有其他方法，像getOwnerProxy和getNonOwnerProxy
}
```

main() 方法创建测试程序，调用其 drive() 方法开始测试。

构造器初始化配对服务人员数据库。

从数据库中获取一个人。

创建一个所有者代理。

调用 getter。

然后调用 setter。

试着改变评分。

这应该是行不通的！

现在创建一个非所有者代理。

调用 getter。

跟着调用 setter。

这应该是行不通的！

试着设置评分。

这应该能行！

运行代码……

```
File Edit Window Help Born2BDynamic
% java MatchMakingTestDrive
Name is Joe Javabean
Interests set from owner proxy
Can't set rating from owner proxy
Rating is 7

Name is Joe Javabean
Can't set interests from non owner proxy
Rating set from non owner proxy
Rating is 5
%
```

我们的所有者代理允许
getting和setting，但不
允许改变极客评分。

我们的非所有者代理只
允许getting，但允许调
用来设置极客评分。

新评分是平均值，取自之前的评分7和
NonOwner代理设定的值3。

没有蠢问题

问：到底“动态代理”动态在哪？是不是指在运行时才将它实例化并设置到一个处理器？

答：不是。代理动态是因为它的类在运行时创建。想一想：代码开始运行前，没有代理类，它是根据需求从你传入的接口集创建的。

问：我的InvocationHandler看起来像一个很奇怪的代理。它没有实现所代理的类的任何方法。

答：这是因为InvocationHandler根本就不是代理，它只是一个帮助代理的类，代理会把方法调用分配给它处理。代理本身是利用静态的Proxy.newProxyInstance()方法在运行时动态创建的。

问：有什么办法能判断某个类是不是代理类呢？

答：有的。代理类有一个静态方法，叫作isProxyClass()。调用这个方法，如果类是一个动态代理类，返回真。除此之外，代理类表现得和其他实现特定接口集的类一样。

问：对于能够传入newProxyInstance()的接口类型，有没有什么限制？

答：是的，有一些限制。首先，应该指出的是，我们总是传给newProxyInstance()一个接口数组，此数组内只能有接口，不能有类。主要限制是，所有非公开接口需要来自同一个包。也不能有带冲突方法名的接口（即，两个接口有一个方法的签名相同）。还有一些比较细微的限制，因此，有时间好好研读一下javadoc中有关动态代理的文件。

★ WHO DOES WHAT? ★

请将下列模式和描述配对：

模式	描述
装饰者	包装另一个对象， 并提供不同的接口。
外观	包装另一个对象， 并提供额外的行为。
代理	包装另一个对象， 并控制对它的访问。
适配	包装许多对象以简化 它们的接口。

代理动物园

欢迎来到对象村动物园！

现在你知道什么是远程代理、虚拟代理和保护代理。在野外，你还会看到这个模式的大量变体。在动物园的代理角，我们收集了捕捉来的许多野生的代理模式，供你研究。

我们的工作尚未完成，但是，确信以后你会在真实世界中看到这个模式的更多变体，因此请你帮帮忙，帮我们编目更多的代理。看看我们现有的收集：



防火墙代理
(Firewall Proxy) 控制
网络资源的访问，保护
主题免于“坏”客户的侵害。

栖息地：常出设于公司的
防火墙系统。

帮助找到栖息地

智能引用代理 (Smart
Reference Proxy) 当主题被引
用时，提供额外的动作，例如
计算一个对象被引用的次数。



缓存代理 (Caching Proxy)
为开销大的操作结果提供暂时
存储：它也允许多个客户共享
结果，以减少计算或网络延迟。

栖息地：常出设在Web服务器代理以及内容管理
与出版系统。

同步代理

(Synchronization Proxy)

在多线程的情况下为主题提供安全访问。



被发现出设于集合中，控制多线程环境中潜在对象集合的同步访问。

帮助找到栖息地

复杂隐藏代理

(Complexity Hiding Proxy)

用来隐藏一个类的复杂集合的复杂度，并控制访问。有时候也称为外观代理 (Facade Proxy)，这不难理解。复杂隐藏代理和外观模式的区别是，代理控制访问，而外观模式只提供了另外的一组接口。

**写入时复制代理**

(Copy-On-Write Proxy)

控制对象的复制，做法是延迟对象的复制，直到客户需要为止。这是虚拟代理的变体。



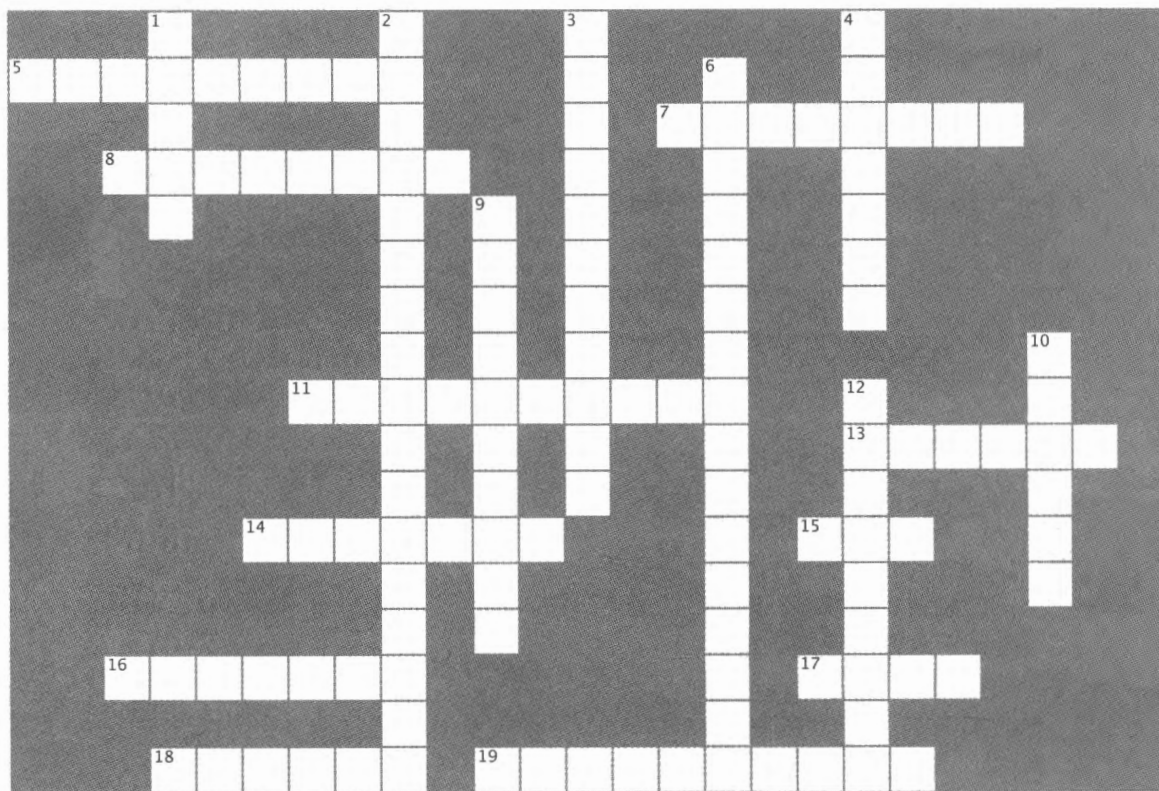
栖息地：出设在Java的CopyOnWriteArrayList附近。

田野笔记：请将你在野外所观察到的其他代理添加在这里：



设计模式填字游戏

这一章很长。在结束前，做做填字游戏休闲一下吧！



横排

5. 所显示的第一个专辑封面的团体（两个词）。
7. 常用于web service的代理（两个词）。
8. RMI中，在服务端接收网络请求的对象。
11. 保护来自未注册调用者的方法调用的代理。
13. 做MCMXC a.D.专辑的团体
14. _____代理类在运行时创建。
15. 学习许多代理变体的地方
16. 专辑浏览器使用这种代理。
17. RMI中，代理被称为这个。
18. 我们拿这些中的一个来学习RMI。
19. 为什么Elroy的约会老是不成功。

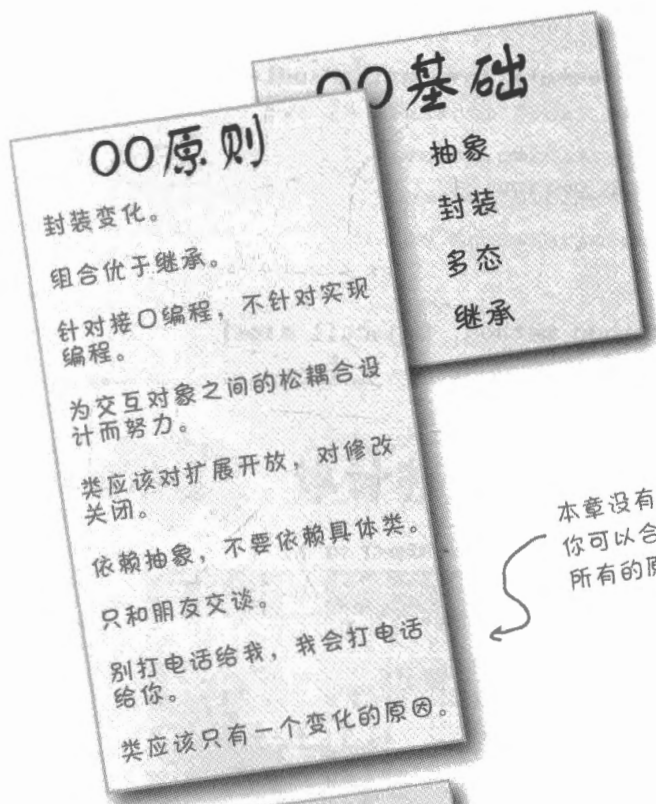
竖排

1. Objectville 配对是为_____服务的。
2. Java的动态代理转发所有请求给它（两个词）。
3. 这个工具作为RMI的查询服务。
4. 站在开销大的对象前面的代理。
6. 远程_____被用于实现糖果机监视器（两个词）。
9. 软件开发人员的经纪人就是这种代理。
10. 我们的第一个错误：糖果机的报告不是_____。
12. 类似于代理，但目标不同。



设计箱内的工具

你的设计工具箱几乎满了，一路下来，你所学会的设计模式，几乎可以解决任何设计问题了。



要点

- 代理模式为另一个对象提供代表，以便控制客户对对象的访问，管理访问的方式有许多种。
- 远程代理管理客户和远程对象之间的交互。
- 虚拟代理控制访问实例化开销大的对象。
- 保护代理基于调用者控制对对象方法的访问。
- 代理模式有许多其他变体，例如：缓存代理、同步代理、防火墙代理、写入时复制代理等。
- 代理在结构上类似装饰者，但是目的不同。
- 装饰者模式为对象加上行为，而代理则是控制访问。
- Java内建的代理支持，可以根据需要建立动态代理类，并将所有调用分配到所选的处理器。
- 和其他的包装者一样，代理会造成你的设计中类和对象的数目增加。



练习答案

NonOwnerInvocationHandler的运作方式就像OwnerInvocationHandler，除了这些区别，它允许调用setGeekRating()而且不允许调用任何其他set方法。下面是我们的解答：

```
import java.lang.reflect.*;

public class NonOwnerInvocationHandler implements InvocationHandler {
    Person person;

    public NonOwnerInvocationHandler(Person person) {
        this.person = person;
    }

    public Object invoke(Object proxy, Method method, Object[] args)
        throws IllegalAccessException {

        try {
            if (method.getName().startsWith("get")) {
                return method.invoke(person, args);
            } else if (method.getName().equals("setGeekRating")) {
                return method.invoke(person, args);
            } else if (method.getName().startsWith("set")) {
                throw new IllegalAccessException();
            }
        } catch (InvocationTargetException e) {
            e.printStackTrace();
        }
        return null;
    }
}
```



设计谜题解答

ImageProxy类看起来有两个状态，由条件语句控制。你能否用另一个模式清理这样的代码？你要如何重新设计ImageProxy？

使用状态模式：实现两个状态ImageLoaded和ImageNotLoaded。然后把来自if语句的代码放进各自的状态中。从ImageNotLoaded状态开始，一旦ImageIcon取回，就转换到ImageLoaded状态。



削尖你的铅笔

答案

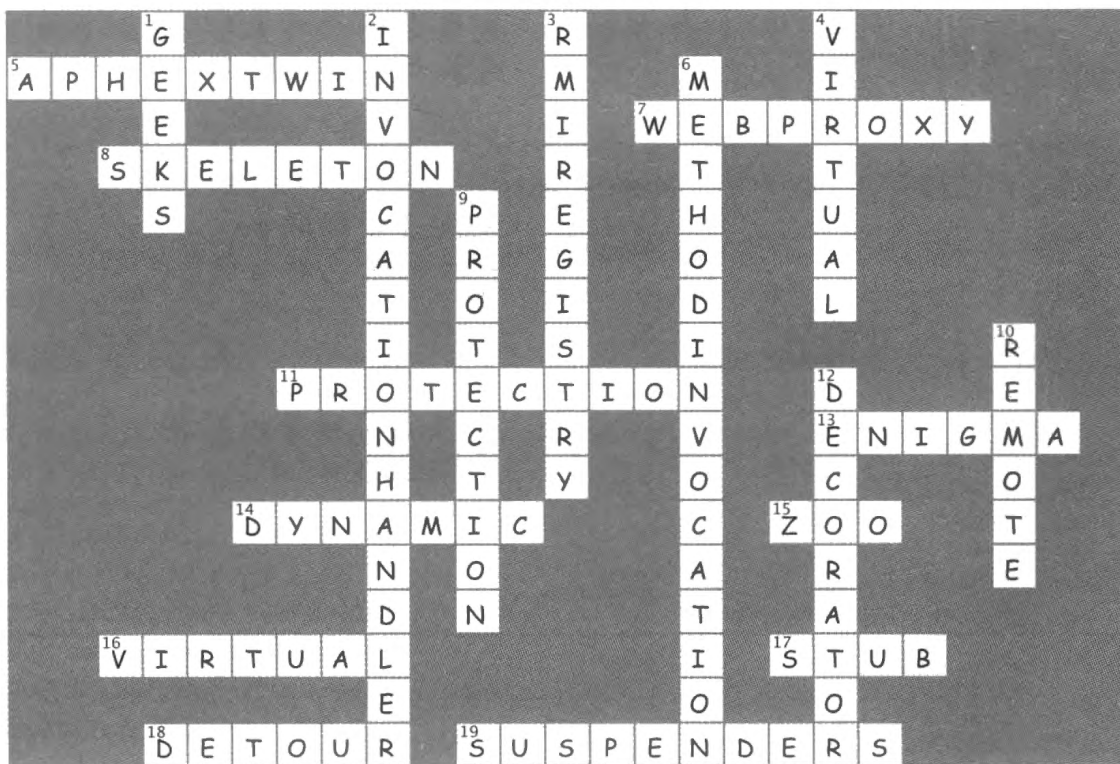
虽然有一点复杂，但是创建动态代理所需要的代码其实不多。请你编写 `getNonOwnerProxy()`，该方法为 `NonOwnerInvocationHandler` 返回代理。
以下是解答：

```
Person getNonOwnerProxy(Person person) {

    return (Person) Proxy.newProxyInstance(
        person.getClass().getClassLoader(),
        person.getClass().getInterfaces(),
        new NonOwnerInvocationHandler(person));
}
```



设计模式填字游戏解答



★ WHO DOES WHAT? ★
答案

请将下列模式和描述配对：

模式

描述

装饰者

包装另一个对象，
并提供不同的接口。

外观

包装另一个对象，
并提供额外的行为。

代理

包装另一个对象，
并控制对它的访问。

适配

包装许多对象以简化
它们的接口。



待烘焙
代码

音乐专辑封面浏览器的代码

```
package headfirst.designpatterns.proxy.virtualproxy;

import java.net.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.util.*;

public class ImageProxyTestDrive {
    ImageComponent imageComponent;
    JFrame frame = new JFrame("Album Cover Viewer");
    JMenuBar menuBar;
    JMenu menu;
    Hashtable<String, String> albums = new Hashtable<String, String>();

    public static void main (String[] args) throws Exception {
        ImageProxyTestDrive testDrive = new ImageProxyTestDrive();
    }

    public ImageProxyTestDrive() throws Exception{
        albums.put("Buddha Bar", "http://images.amazon.com/images/P/B00009XBYK.01.LZZZZZZZ.
jpg");
        albums.put("Ima", "http://images.amazon.com/images/P/B000005IRM.01.LZZZZZZZ.jpg");
        albums.put("Karma", "http://images.amazon.com/images/P/B000005DCB.01.LZZZZZZZ.
gif");
        albums.put("MCMXC a.D.", "http://images.amazon.com/images/P/B000002URV.01.
LZZZZZZZ.jpg");
        albums.put("Northern Exposure", "http://images.amazon.com/images/P/B000003SFN.01.
LZZZZZZZ.jpg");
        albums.put("Selected Ambient Works, Vol. 2", "http://images.amazon.com/images/P/
B000002MNZ.01.LZZZZZZZ.jpg");

        URL initialURL = new URL((String)albums.get("Selected Ambient Works, Vol. 2"));
        menuBar = new JMenuBar();
        menu = new JMenu("Favorite Albums");
        menuBar.add(menu);
```



待烘焙 代码

音乐专辑封面浏览器的代码 继续……

```

frame.setJMenuBar(menuBar);

for(Enumeration e = albums.keys(); e.hasMoreElements();) {
    String name = (String)e.nextElement();
    JMenuItem menuItem = new JMenuItem(name);
    menu.add(menuItem);
    menuItem.addActionListener(event -> {
        imageComponent.setIcon(
            new ImageProxy(getAlbumUrl(event.getActionCommand())));
        frame.repaint();
    });
}

// 设置框架和菜单

Icon icon = new ImageProxy(initialURL);
imageComponent = new ImageComponent(icon);
frame.getContentPane().add(imageComponent);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setSize(800,600);
frame.setVisible(true);

}

URL getAlbumUrl(String name) {
    try {
        return new URL((String)albums.get(name));
    } catch (MalformedURLException e) {
        e.printStackTrace();
        return null;
    }
}
}

```



待烘焙
代码

音乐专辑封面浏览器的代码 继续……

```
package headfirst.designpatterns.proxy.virtualproxy;

import java.net.*;
import java.awt.*;
import javax.swing.*;

class ImageProxy implements Icon {
    volatile ImageIcon imageIcon;
    final URL imageURL;
    Thread retrievalThread;
    boolean retrieving = false;

    public ImageProxy(URL url) { imageURL = url; }

    public int getIconWidth() {
        if (imageIcon != null) {
            return imageIcon.getIconWidth();
        } else {
            return 800;
        }
    }

    public int getIconHeight() {
        if (imageIcon != null) {
            return imageIcon.getIconHeight();
        } else {
            return 600;
        }
    }

    synchronized void setImageIcon(ImageIcon imageIcon) {
        this.imageIcon = imageIcon;
    }

    public void paintIcon(final Component c, Graphics g, int x, int y) {
        if (imageIcon != null) {
            imageIcon.paintIcon(c, g, x, y);
        } else {
            g.drawString("Loading album cover, please wait...", x+300, y+190);
            if (!retrieving) {
                retrieving = true;
            }
        }
    }
}
```



待烘焙 代码

音乐专辑封面浏览器的代码 继续……

```
        retrievalThread = new Thread(new Runnable() {
            public void run() {
                try {
                    setImageIcon(new ImageIcon(imageURL, "Album Cover"));
                    c.repaint();
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
        retrievalThread.start();
    }
}

package headfirst.designpatterns.proxy.virtualproxy;

import java.awt.*;
import javax.swing.*;

class ImageComponent extends JComponent {
    private Icon icon;

    public ImageComponent(Icon icon) {
        this.icon = icon;
    }

    public void setIcon(Icon icon) {
        this.icon = icon;
    }

    public void paintComponent(Graphics g) {
        super.paintComponent(g);
        int w = icon.getIconWidth();
        int h = icon.getIconHeight();
        int x = (800 - w)/2;
        int y = (600 - h)/2;
        icon.paintIcon(this, g, x, y);
    }
}
```